Odin: Rethinking Congestion Control under All-to-All Traffic

Wanchun Jiang
Central South University
Changsha, China

Yujie Hu*
Central South University
Changsha, China

Kai Wang
Central South University
Changsha, China

Xiao Han
Central South University
Changsha, China

Haoran Wei
ByteDance
Beijing, China

Yong Zhang
ByteDance
Beijing, China

Jianxi Ye
ByteDance
Beijing, China

Huichen Dai†
ByteDance
Beijing, China

Abstract

All-to-All (A2A) communication dominates the training and inference time of large language models, especially under the Mixture-of-Experts (MoE) architecture where expert dispatch induces dense concurrent flows. We observe that A2A traffic consistently builds up large total buffer occupancy (TBO) at switches. Analyzing the accompanying counterintuitive phenomena, we reveal that the A2A traffic saturates ingress bandwidth, interferes with rate adjustment of congestion control (CC) through sparse congestion signals and rate inflation, and causes ingress-egress bandwidth mismatch. Unaware of these issues, existing CC algorithms would be easy to trigger PFC with large TBO, suffer from ineffectual congestion response, and lead to throughput loss. Based on these insights, we propose Odin to enhance CC under A2A traffic. Odin suppresses large TBO based on the switch-level TBO signals, proactively allocate ingress bandwidth to eliminate rate inflation, and make bandwidth compensation for ingress-egress matching. Experiments and simulations show that Odin achieves low TBO and high throughput under A2A traffic, accelerating the training process. Meanwhile, it keeps strong performance under non-A2A traffic.

CCS Concepts

• **Networks** → **Transport protocols**; **Data center networks**; *Network protocol design.*

Keywords

Congestion Control, All-to-All Traffic, Large Total Buffer Occupancy, Rate Inflation, Ingress-Egress Bandwidth Match

ACM Reference Format:
Wanchun Jiang, Yujie Hu, Kai Wang, Xiao Han, Haoran Wei, Yong Zhang, Jianxi Ye, and Huichen Dai. 2026. Odin: Rethinking Congestion Control under All-to-All Traffic. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26), August 17–21, 2026, Denver, CO, USA.* ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/3789240.3829164

*This work was done while Yujie Hu was an intern at ByteDance
†Corresponding author.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
SIGCOMM '26, August 17–21, 2026, Denver, CO, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2467-1/26/08
https://doi.org/10.1145/3789240.3829164

1 Introduction

All-to-All (A2A) communication has become essential in distributed training and inference [1–3], driven by the widespread adoption of large language models (LLM) and, in particular, the Mixture-of-Experts (MoE) architecture such as DeepSeek [4]. In the distributed training process, A2A communication accounts for 35%~70% of training time [5–7]. Therefore, the efficiency of A2A communication has become a decisive factor for the performance of model training and inference.

Congestion control (CC) is the key to improving network performance. Existing CC algorithms for datacenter network work well for traffic patterns such as incast [8] and permutation [9], where congestion typically concentrates at a few bottleneck egress ports. Different from these traffic patterns, A2A traffic saturates all ingress and egress ports simultaneously and induces a large total buffer occupancy (TBO) on the switch [10], making existing CC algorithms struggle for expected network performance. In other words, A2A traffic brings new challenges to CC.

In this work, we observe three counterintuitive phenomena accompanying large TBO and reveal the underlying reasons. Firstly, Priority Flow Control (PFC) [11] may be triggered prior to Explicit Congestion Notification (ECN), since the dynamic PFC threshold under large TBO may become smaller than the static ECN threshold. This large TBO is usually accumulated by many small port buffer occupancies due to the bandwidth saturation of all ports under A2A traffic. Secondly, there are tens or hundreds of round-trip time (RTT) from the onset of CC rate reduction to the time point of corresponding buffer occupancy reduction. This is due to the sparse rate adjustments and sending rate inflation generated by the intense queue pair (QP) competition at NIC under A2A traffic. Here, a QP of the Remote Direct Memory Access (RDMA) protocol [12] corresponds to a flow. Thirdly, the growth of TBO is accelerated when CC takes effect. This is because the bandwidth relinquished by one QP is immediately preempted by other QPs at the same NIC due to the sending rate inflation.

Furthermore, we rethink CC under A2A traffic and obtain the following insights. First, A2A traffic makes TBO critical. Unaware of the switch-level TBO, existing CC algorithms may frequently

trigger PFC. Second, A2A traffic leads to interference on the rate adjustment, including sparse rate adjustment and rate inflation. Unaware of the interference, existing CC algorithms may have slow congestion response and ineffectual execution of its calculated rate. Third, A2A traffic leads to ingress-egress bandwidth mismatch. Due to the exclusive focus on egress bandwidth, existing CC algorithms are unaware of this mismatch and thus may result in link underutilization and long completion time. In sum, A2A traffic narrows the design space of CC, and it is insufficient to address the above issues by merely tuning the existing CC algorithms (§3).

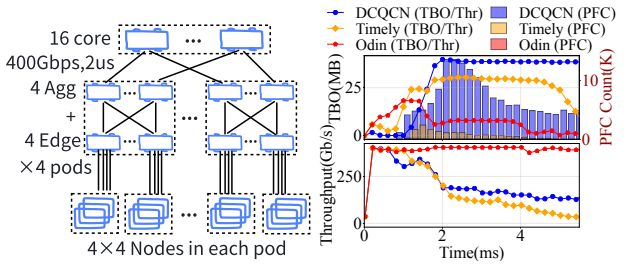
Therefore, we suggest a systematic, external enhancement to the CC framework for A2A traffic, and propose Odin, which enhances existing CC from three aspects. First, Odin uses a specialized TBO signal marked by switches to proactively suppress the large TBO, thereby preventing frequent PFC triggering. Second, Odin regards the degree of competition at NIC as a congestion signal to perceive interference on rate adjustment. By coordinating CC calculations with proactive bandwidth allocation at the NIC, Odin eliminates the rate inflation and thus ensures effectual execution of the fast rate adjustment. Third, Odin identifies ingress–egress bandwidth mismatch by jointly reasoning about the network congestion signals and the virtual congestion signals at NICs. Building on this, Odin selectively compensates ingress bandwidth to match the ingress-egress bandwidth, thereby maximizing link utilization and accelerating A2A traffic completion. Collectively, these design choices are systematically integrated into a classic CC paradigm without influencing the paradigm's fast congestion response and fairness. As a result, Odin addresses the performance degradation issues under A2A traffic, while keeping good performance under non-A2A traffic.

We implement Odin on the NIC with DPDK, and enable TBO support on Tofino switches by one arithmetic logic unit. Testbed experiments conducted under A2A traffic demonstrates that Odin reduces TBO by 78.7% over DCQCN [13] and improves throughput by 1.75× over HPCC [14]. Moreover, large-scale simulations assess Odin under various scenarios. Under A2A traffic with 512 nodes, Odin maintains high throughput and reduces the TBO by 88%, 78%, 55%, 80%, and 76% compared to DCQCN, Timely [15], HPCC, PowerTCP [16], and Poseidon [17], respectively. Moreover, Odin reduces the completion time of MoE model training by 16%, 40%, 13%, 3%, and 19%, compared to DCQCN, Timely, HPCC, PowerTCP, and Poseidon, respectively. Besides, the number of PFC pauses is reduced by 18%, 24%, 8%, and 14%, compared to DCQCN, Timely, PowerTCP, and Poseidon, respectively. Under non-A2A traffic, including AlexNet [18] model training and classic datacenter workloads, Odin also achieves good performance, which is comparable to existing INT-based CC algorithms.

2 Background and Motivation

2.1 Background

Currently, the MoE architecture is widely adopted to reduce the training and inference costs of LLMs, since its computation grows sublinearly with model size by activating a small subset of experts for each token [19–23]. In MoE pipelines, tokens are routed between attention blocks and expert layers, inducing the all-to-all (A2A) communication. In an expert-parallel group with N nodes/GPUs,



(a) Topology (b) TBO and Average Throughput of Leaf Switch and PFC number

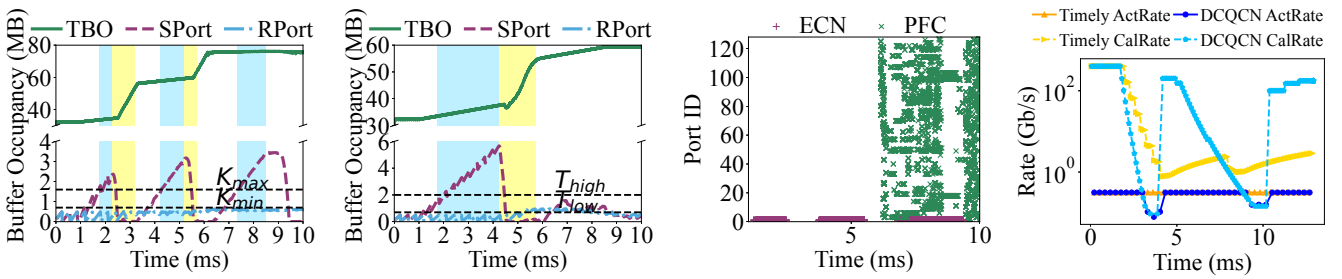
Figure 1: Large total buffer occupancy, throughput loss and large amounts of PFC.

A2A entails each participant communicating with every other participant, creating $N * (N - 1)$ communicating pairs in each exchange round. Empirically, the A2A traffic occupies a substantial portion of model training time, e.g., 38.5%~68.4% as reported in Janus [5], 35.4%~39.5% in Lina [6], and 50.8%~58.8% in ScheMoE [7]. This is different from traditional datacenter networks and high-performance computing [24, 25], where A2A traffic has not received much attention due to its small size and scale. In sum, the A2A traffic should be taken seriously for the efficiency of training and inference in LLM.

Furthermore, the traffic greatly influences the performance of CC. The state-of-the-art CC algorithm, DCQCN [13], widely deployed in Mellanox NICs, employs explicit congestion notification (ECN) to show whether the buffer occupancy exceeds thresholds K_{min} and K_{max}. These thresholds are always set static values [26]. Similarly, Timely [15], employed by Google, distinguishes congestion based on whether RTT exceeds thresholds T_{low} and T_{high}. Moreover, Alibaba suggests employing the In-Network Telemetry (INT) in switches for congestion measurement [14]. Leveraging INT, HPCC [14] and PowerTCP [16] try to maintain the buffer occupancy close to 0. In sum, all these CC algorithms adjust the sending rate according to their measured states of the bottleneck switch port. In addition, to ensure losslessness for RDMA, the Priority-based Flow Control (PFC) is employed [11]. When the dynamic threshold is exceeded, the switch sends PFC pause frames to halt upstream data transmission on that priority. While preventing loss, PFC may induce head-of-line blocking, congestion spreading, and deadlocks, severely degrading network performance [27–30]. Consequently, CC should reduce the frequency of triggering PFC. All these mechanisms are designed for general traffic patterns and do not explicitly account for the distinctive characteristics of A2A traffic.

2.2 Large Total Buffer Occupancy

In our real-world clusters, we observe that A2A traffic causes large total buffer occupancy (TBO) at the bottleneck switch, leading to significant performance degradation. A similar issue has also been reported in Meta clusters [10]. To explore the underlying reason, we reproduce the issue in simulations.



(a) Buffer occupancy of DCQCN (b) Buffer occupancy of Timely (c) PFC and ECN of DCQCN. (d) Rate Adjustment

Figure 2: Counterintuitive Phenomena under A2A Traffic.

2.2.1 Observation. Simulations are conducted based on NS3 [31] with the 128-nodes fat-tree topology shown in Fig. 1(a). A2A traffic starts concurrently from all QPs. Detailed configuration is in §6.1. The TBO at one leaf switch, the total number of PFC pause across the network, and the average throughput, i.e., the average receiving rates across all nodes, are shown in Fig. 1(b). Both DCQCN and Timely suffer from large TBO. Correspondingly, PFC is triggered frequently with DCQCN and the throughput loss is heavy. Meanwhile, Timely also suffer from severe throughput loss.

To reveal the underlying reasons for the large TBO, the simulation is repeated in a simpler scenario, where all switches in Fig. 1(a) are replaced by a single switch with 128 ports. Moreover, we let 127 nodes transmit A2A traffic, and the rest one node sends 8KB short flows every $10us$ to another node participating in A2A since $1ms$. The egress port for short flow at the switch is referred to as SPort, and we randomly select another port as a comparison, referred to as RPort. As shown in Fig. 2(a) and Fig. 2(b), the large TBO of the switch still occurs similar to Fig. 1(b). Accompanying the large TBO, counterintuitive phenomena are observed.

[Phenomenon 1] PFC is triggered prior to ECN due to large TBO. Intuitively, PFC should be triggered after ECN as the PFC threshold is expected to be larger than ECN threshold. However, as shown in Fig. 2(c), PFC is triggered frequently on almost all switch ports, regardless of whether ECN is triggered. The direct reason is that the PFC threshold is dynamically set according to the remaining buffer occupancy [32] and becomes smaller than the static ECN threshold due to the large TBO [13]. This is confirmed by Fig. 2(a), where the buffer occupancy of RPort is smaller than the ECN threshold but the TBO is large before $5ms$. Note that the large TBO is not solely owed to the large buffer occupancy of SPort, but mainly accumulated among many other ports with small buffer occupancy similar to RPort.

[Phenomenon 2] There are tens or hundreds of RTT from the onset of CC rate reduction to the time point of corresponding buffer occupancy reduction. In the blue region of Fig. 2(a) and Fig. 2(b), the buffer occupancy of SPort exceeds the thresholds to a huge extent for several milliseconds. It indicates that both DCQCN and Timely take several milliseconds to decrease the sending rate for buffer occupancy elimination. This is a contradiction to the intuitive understanding that the time to eliminate the buffer occupancy of SPort is on the same order of magnitude as RTTs, i.e., several microseconds.

[Phenomenon 3] The buildup of TBO is accelerated when CC takes effect. This phenomenon is shown in the yellow region of Fig. 2(a) and Fig. 2(b). TBO doesn't decrease with the reduction of the buffer occupancy of SPort, but increases fast. It indicates that the buffer occupancy of many other ports increases slowly and the acceleration is owed to the large number of ports. Besides, the buffer occupancy of RPort is always below the thresholds.

2.2.2 Underlying Reasons. We analyze the simulation data and reveal the underlying reasons for the above phenomena.

The reason for **Phenomenon 1** is that **All-to-all traffic saturates all ingress bandwidth and thus leads to the accumulated large TBO.** Specifically, because CC starts at line rate, many packets are injected from all the 127 ingress ports, but may not be forwarded out the switch in time. This corresponds to the large TBO after initialization in Fig. 2(a) and Fig. 2(b). Moreover, the value of this large initial TBO is directly proportional to the number of ports under A2A traffic and evenly contributed by all ports, as analyzed in Appendix A. After initialization, the total injection rates of 127 ingress ports would be $127b$ and the total drain rates of 127 egress ports is not larger than $127b$, where b is the link capacity $400Gbps$. Therefore, the large initial TBO increases slowly during time $[0ms, 1ms]$. In other words, all ingress bandwidths are saturated by the A2A traffic. After 1 ms, the arrival of short flows further increases the buffer occupancy of SPort. Meanwhile, the buffer occupancy also grows on ports that do not directly share traffic with the short flows. This indicates that all ingress bandwidth remains saturated under A2A traffic. Therefore, the TBO is accumulated to the large value which makes the dynamic PFC threshold smaller than the static ECN threshold.

The reason for **Phenomenon 2** is that **the sending rate is inflated and the rate adjustment becomes sparse due to the large number of QPs under A2A traffic.** In simulations, 5 QPs are configured between each pair of nodes, and each node has $126*5=630$ QPs. Thus, the average actual sending rate of each QP is $400Gbps/630 \approx 635Mbps$. On initialization, the calculated rate, i.e., the sending rate set by CC for each QP, is $400Gbps$, greatly larger than the actual sending rate. This phenomenon is referred to as *rate inflation*. What's worse, subsequent rate adjustments in CC are based on the calculated rate, and the actual sending rate determined by QP competition at NIC is not aware. Therefore, the rate inflation

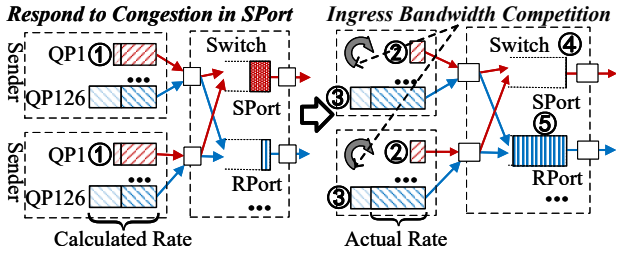


Figure 3: The buildup of TBO is accelerated by CC.

persists for a long time as shown in Fig. 2(d). Besides, the average packet sending interval, which is about $4KB/635Mbps \approx 50us$, would be greatly larger than the base RTT $8us$. Accordingly, the intervals of receiving ACKs and the corresponding rate adjustment become sparse, as shown in Fig. 2(d). Due to the sparse rate adjustment, CC requires a long time to make the calculated rate close to the actual sending rate. For example, to reduce the calculated rate from $400Gbps$ to below $635Mbps$, DCQCN requires at least 10 rounds of rate adjustment, i.e., about $50us * 10 = 0.5ms$. These are why both DCQCN and Timely take several milliseconds to eliminate the buffer occupancy of SPort in Fig. 2(a) and (b).

The reason for **Phenomenon 3** is that **the relinquished bandwidth of a QP would be preempted by other QPs at this NIC due to the inflated sending rate.** As shown in Fig. 3, when SPort becomes congested, the QPs sharing SPort decrease their calculated rates, as shown by ①. The actual sending rate of this QP is reduced after the calculated rate becomes smaller than it as shown by ②. Meanwhile, the ingress bandwidth relinquished due to the rate decrease of this QP would be preempted by other QPs at the same NIC, because their calculated rates are still inflated. Thus, the actual sending rate of these non-congested QPs will be slightly increased, although their calculated rate remains unchanged, as shown in ③. Consequently, the buffer occupancy of SPort would be reduced and may lead to link under-utilization, as shown in ④. Meanwhile, the buffer occupancy of other ports would gradually increase as shown in ⑤. This is also reflected by the yellow regions of Fig. 2(a) and (b). Moreover, the total sending rate of each ingress port remains at $400Gbps$. This is why the TBO accumulation is accelerated when the buffer occupancy of SPort is reduced by CC.

In total, the underlying reasons for the above three counterintuitive phenomena indicate that the A2A traffic and improper CC algorithm lead to the large TBO together. Specifically, **A2A traffic** saturates the bandwidth of all ingress ports, generating the initial large and growing TBO at first. Second, **the QP itself**, whose rate is inflated and rate adjustment is sparse, suffers from slow response to TBO. Third, **the interference among QPs** at the same NIC, i.e., the bandwidth preemption at the NIC, leads to the sharp growth of large TBO. Notably, the three phenomena help to uncover the underlying reasons, but they do not necessarily occur simultaneously under every network configuration, as detailed in Appendix D.1.

3 Design Space

In brief, the above insight suggests that CC should be enhanced under A2A traffic from three aspects as shown in Fig. 4. Firstly, existing

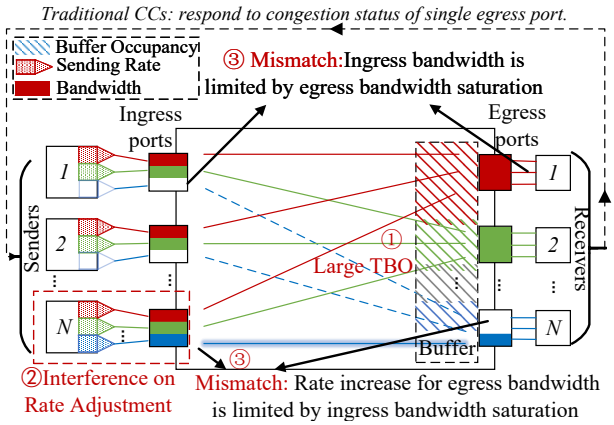


Figure 4: Design Space of CC under A2A Traffic.

CC algorithms focus solely on the buffer occupancy of the congested egress port and are unaware of the **large TBO**. Accordingly, unnecessary PFC triggering and accompanying throughput loss would occur under A2A traffic. Secondly, existing CC algorithms are unaware of the **interference on rate adjustment** caused by massive QP concurrency at the ingress links. Accordingly, the congestion responsiveness becomes sparse and may not be executed in time under A2A traffic. Thirdly, existing CC algorithms are unaware of the **ingress-egress bandwidth mismatching**, which often arises from the improper competition of the relinquished bandwidth such as the ingress bandwidth preemption discussed in §2.2.2. Accordingly, the mismatching leads to link under-utilization and long-tail flow completion of A2A traffic. Overall, the design space of CC is narrowed under A2A traffic, and makes the enhancements to existing CC algorithms necessary as discussed below.

3.1 Switch-Level TBO Awareness

A2A traffic makes TBO critical in the face of the limited buffer space of modern switches. As shown in Fig. 4, different from traditional congestion scenarios, where the queue length of congested egress port is large, the queue length of each egress port may be small under A2A traffic yet the TBO is large. The large TBO would make it easy to trigger PFC and thus leads to severe throughput degradation.

However, existing CC algorithms are unaware of switch-level large TBO and only focus on port signals. Even though lowering the expected port buffer occupancy, the performance degradation cannot be solved. For example, we lower the ECN thresholds of DCQCN [13] to 50KB/100KB, referred to as DCQCN+. The simulations of Fig. 2 are repeated. As shown in Fig. 5, DCQCN+ still has a large TBO of about $70MB$, and the queue length distributed to each port, $70MB/128ports \approx 547KB$, far exceeds the ECN thresholds. The underlying reason for the large TBO is that although DCQCN decreases rates upon receiving ECN signals, it also spontaneously increases the sending rate by a huge amount during the large intervals of receiving ECN signals. This issue is hard to solve by tuning DCQCN, as confirmed in [10]. Therefore, the traditional wisdom of *spontaneous rate increase does not suit CC under A2A traffic.* On the other hand, HPCC and PowerTCP aim to maintain extremely small buffer occupancy at each port, but the actual TBO are around $20MB$

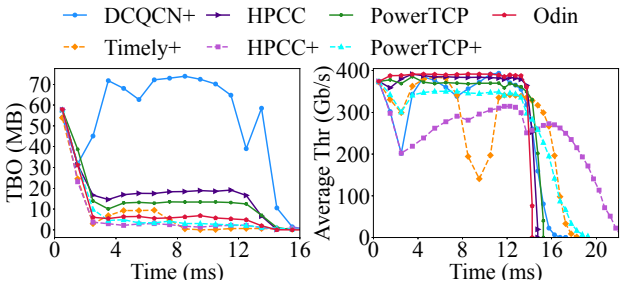


Figure 5: TBO and Throughput of CC algorithms.

in Fig. 5. This is because their per-port buffer occupancy grows with their fixed additive increase (AI) factors and the number of QPs sharing this port [33], thereby increasing the aggregated TBO. Furthermore, to make the per-port buffer occupancy independent of the QP numbers, we set the AI factors of HPCC and PowerTCP to 0, named as HPCC+ and PowerTCP+, respectively. Although the TBO is suppressed, the completion time of A2A traffic becomes long due to unfairness, as explained in §3.3. Consequently, **the explicit switch-level TBO awareness should be incorporated in the CC design space**.

3.2 Interference on Rate Adjustment

A2A traffic may lead to interference on the rate adjustment of CC algorithms, as depicted in the ingress-side of Fig. 4. In detail, the interference includes both the sparse and slow rate adjustment and the inflated rates, which lead to ineffectual rate adjustment as discussed in §2.2.2.

However, existing CC algorithms are unaware of such interference on rate adjustment under A2A traffic, which would degrade their performance. For example, we have lowered the delay thresholds of Timely to 9us/10us to reduce TBO, referred to as Timely+. Although Timely+ indeed achieves a small TBO under A2A traffic as shown in Fig. 5, it suffers from severe throughput loss, prolonging the completion time of A2A traffic by about 20% compared to the best one. Data analysis reveals two primary causes. First, the probing rate adjustment of Timely+ is slow due to the sparse rate adjustment chances, similar to explanations in [14]. Second, its rate adjustment always becomes ineffectual due to rate inflation. Hence, due to the interference on rate adjustment, the traditional wisdom of *probing rate adjustment of CC is not suitable under A2A traffic*. In contrast, HPCC+ and PowerTCP+ have fast rate adjustment [14, 16] and thus a medium frequency of rate inflation according to the underlying data analysis. In total, **the awareness of the ingress-side interference on the rate adjustment should be incorporated in the CC design space.**

3.3 Ingress-Egress Bandwidth Mismatching

A2A traffic may lead to the ingress-egress bandwidth mismatching as shown in Fig. 4. Specifically, when QPs at ingress port 1 and 2 finish, the following two-fold mismatching occurs. On the one hand, although there is idle bandwidth at ingress ports 1 and 2, CC would not increase rates (red/green) because the bandwidths of egress ports 1 and 2 are saturated. On the other hand, although CC

Table 1: Existing CC algorithms under A2A traffic.

CC algorithms	TBO	Fast Adjustment	Rate Inflation	Bandwidth Matching
DCQCN	large	medium	frequent	poor
DCQCN+	large	medium	frequent	poor
Timely	large	medium	frequent	poor
Timely+	low	medium	frequent	poor
HPCC/PowerTCP	medium	good	medium	poor
HPCC+/PowerTCP+	low	good	medium	poor
Odin	low	good	infrequent	good

is aware of the idle bandwidth at egress port 3, the rate increase (blue) at ingress port 3 would be ineffectual because there is no idle ingress bandwidth. Consequently, the mismatching would result in both ingress and egress links utilization loss, prolonging the completion time of A2A traffic. Besides, these bandwidth mismatching is determined by both the bandwidth competition at ingress link, which is significant under A2A traffic as discussed in §2.2, and the bandwidth allocation of CC at the congested egress link.

However, the existing CC is unaware of the ingress-egress bandwidth mismatching, as it solely focuses on the bandwidth allocation of the congested egress link. This is why the unfair egress bandwidth allocation makes HPCC+ and PowerTCP+ have a long completion time of A2A traffic in Fig. 5. Furthermore, we theoretically depict the inevitability of the long completion time caused by existing CC. Firstly, define set $S(t) = \{QP_{ij}(t)\}$, where the stochastic variable $QP_{ij}(t)$ stands for the average calculated rate $r_{ij}(t)$ of the QP from i^{th} ingress port to j^{th} egress port at time t. Here $r_{ij}(t)$ is assumed to remain unchanged in the stable state as in [34]. When a QP finishes, CC would adjust the calculated rates of the remaining active QPs, named as set $A(t) \subseteq S(t)$, to converge towards a new stable state again within time δt. Specifically, QPs sharing the same egress bottleneck with the finished QP in $A(t)$ will increase their calculated rates under the subsequent new stable state to acquire the relinquished egress bandwidth. Meanwhile, the rest QPs in $A(t)$ would keep their calculated rates unchanged. In total, existing CC algorithms exhibit the following feature.

Feature 1. *When a QP finishes, for any i, j, $QP_{ij}(t) \in A(t)$, $QP_{ij}(t+\delta t) \in A(t+\delta t)$, there is $r_{ij}(t+\delta t) \geq r_{ij}(t)$.*

Feature 1 indicates that existing CC algorithms only focus on improving the egress utilization, and would not proactively reduce their rates to avoid competition with other QPs at the ingress port. On this basis, we have the following theorem referring to Hall's theorem [35].

Theorem 1. *Under A2A traffic, due to the Feature 1 of existing CC algorithms, there exists $A(t)$ for which the ingress-egress bandwidth matching cannot be satisfied. This implies that the ingress-egress bandwidth mismatching is inevitable.*

Detailed proof is shown in Appendix B. The ingress-egress bandwidth mismatching may prolong the A2A completion time and thus the traditional wisdom of *focusing solely on bandwidth allocation of the congested egress link is not suitable for CC under A2A traffic*. In

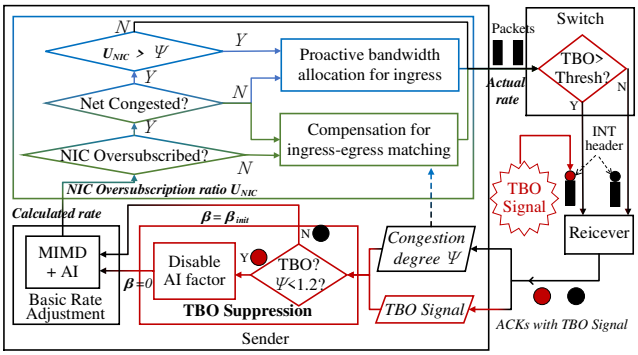


Figure 6: Framework of Odin.

contrast, **the awareness of the mismatching between ingress and egress bandwidth should be explicitly incorporated into the design space of CC.** What's more, for preventing the occurrence of bandwidth mismatching, **the fairness of CC becomes more important under A2A traffic**.

In summary, under the newly identified design space introduced by A2A traffic, existing CC algorithms exhibit different issues in the respects of TBO, fast rate adjustment, rate inflation, and bandwidth matching, as shown in Table 1. Moreover, it is insufficient to bridge these gaps by merely tuning the existing CC algorithms. Therefore, we suggest a systematic, external enhancement to the CC framework under A2A traffic.

4 Design of Odin

To address the above issues, we propose Odin. Odin serves as a concrete realization to demonstrate how the enhancements to existing CC algorithms can be systematically integrated to address the unique challenges of A2A traffic.

4.1 Framework

Following the above design space, Odin makes three enhancement choices to existing CC, including TBO suppression (§4.2), proactive bandwidth allocation (§4.3), and ingress-egress bandwidth matching (§4.4). The framework of Odin is shown in Fig. 6.

Firstly, besides the single-port signals used in existing CC algorithms, Odin incorporates a specialized TBO signal marked by the switch to perceive congestion from a switch-level perspective. Based on the awareness of TBO, Odin suppresses the large TBO to prevent persistent buffer exhaustion and reduce the frequency of triggering PFC.

Secondly, Odin utilizes the NIC oversubscription ratio, i.e., the total degree of rate inflation at the NIC, to perceive the interference on rate adjustment. Based on the awareness, Odin coordinates CC calculations with proactive bandwidth allocation at the NIC to align the calculated rate with the actual rate. Consequently, the rate inflation becomes infrequent, enabling fast rate adjustment.

Thirdly, Odin identifies ingress-egress bandwidth mismatching by comparing the NIC oversubscription ratio with the network congestion degree. Based on the awareness, Odin chooses to compensate the ingress bandwidth to break the Feature 1 of the existing CC.

Algorithm 1: Pseudo code of Odin.

```
1  Function OnAckArrival():
2      (q, μ) ← Parse(Ack)
3      ψ = (μ·τ+q)/(b·τ) ;                    // Network congestion degree Eq. (2)
4      if Ack.TBO && ψ < 1.2                  // TBO Suppression
5          β ← 0 ;                            // Disable AI factor
6      else
7          β ← β_init ;                       // Default AI factor
8      R_i ← R_i · 1/ψ · η + β ;              // Eq. (3)
9      U_NIC = γ · Σ_i R_i / B_NIC + (1 − γ) · U_NIC ;   // Eq. (4)
10     if U_NIC > 1
11         if ψ > 1 && U_NIC > ψ              //Eliminate rate inflation
12             R_i ← R_i · (γ/U_NIC + 1 − γ) ;          // Eq. (5)
13         else if ψ < 1                      //Improve egress utilization
14             R_i ← R_i · (γ/U_NIC + 1 − γ) + β_init ;  // Eq. (5) (6)
15     else if U_NIC < 0.9                    //Improve ingress utilization
16         R_i ← R_i + β_init ;               // Eq. (6)
```

Therefore, Odin can match the ingress-egress bandwidth, avoiding under-utilization and accelerating the completion of A2A traffic.

Collectively, the above design choices are systematically integrated into a classic CC paradigm, which adopts MIMD rules for fast rate adjustment and additive increase (AI) rules for fairness, similar to [14, 16]. As a result, Odin not only performs well under A2A traffic, but also keeps good performance under non-A2A workloads, since these design choices are primarily activated under A2A traffic. For ease of understanding, the pseudo-code of Odin is shown in Algorithm 1.

4.2 TBO Suppression

Odin collects explicit TBO signals from the switch for TBO suppression. Specifically, switch sets a one-bit TBO flag when the total buffer occupancy exceeds the following threshold.

$$Th_{TBO} = \alpha_{TBO} \cdot B \tag{1}$$

Here B is the total switch buffer size and α_{TBO} is the expected TBO ratio. The TBO flag is inserted into the INT header, delivered to the receiver, and piggybacked to the sender via ACK. Upon receiving the ACK, the sender parses the TBO flag from the INT header.

Based on the TBO flag, Odin suppresses the large TBO by selectively disabling the AI rules in the classic CC paradigm. If the TBO flag is set and the network congestion degree ψ is not severe, i.e., $Ack.TBO \;\&\&\; \psi < 1.2$, Odin temporarily disables the AI rules by setting the AI factor $\beta = 0$, as shown in Lines 4-5 of Algorithm 1. Odin uses the same calculation as HPCC [14] to estimate the network congestion degree ψ.

$$\psi = \frac{\mu \cdot \tau + q}{b \cdot \tau} \tag{2}$$

Here b is the link bandwidth, τ is the base RTT, q and μ are the buffer occupancy and throughput of the bottleneck port, respectively. When the congestion degree is severe, a large TBO may be caused

by intense congestion on a small number of ports rather than A2A traffic. In this case, Odin keeps the AI factor enabled and relies on the MIMD rate adjustment to suppress port-level congestion while preserving fairness.

Overall, by using the TBO flag as a switch-level buffer occupancy signal, Odin can distinguish whether a large TBO is caused by A2A traffic or by common concurrent congestion. When the large TBO is attributed to A2A traffic, disabling AI ensures that the equilibrium buffer occupancy of bottleneck ports approaches zero, instead of scaling with the number of flows [33]. As a result, the aggregated ingress rate of the switch is reduced to be slightly lower than the aggregated egress rate, allowing TBO to be effectively eliminated.

4.3 Proactive Bandwidth Allocation

To be aware of the interference on rate adjustment, Odin assesses the degree of competition at the NIC and incorporate the proactive bandwidth allocation into the classic MIMD-based rate adjustment. Specifically, the MIMD rule benefits fast rate adjustment, with the calculated rate update as

$$R_i \leftarrow R_i \cdot \frac{1}{\psi} \cdot \eta + \beta \tag{3}$$

Here R_i is the calculated rate of QP_i. η is the expected egress link utilization with a default value of 0.95, and β is the AI factor for fairness, similar to [14]. After each rate adjustment, Odin assesses the competition of all QPs at the NIC and allocates bandwidth according to the capacity of the NIC. To capture the degree of competition at the NIC, Odin defines the NIC oversubscription ratio as the smoothed normalized sum of the calculated sending rates of active QPs.

$$U_{NIC} = \gamma \cdot \frac{\sum_i R_i}{B_{NIC}} + (1-\gamma) \cdot U_{NIC} \tag{4}$$

Here B_{NIC} denotes the NIC bandwidth. U_{NIC} reflects the degree of rate inflation at the NIC, and γ is an EWMA factor.

Subsequently, Odin makes proactive bandwidth allocation according to both the network congestion degree ψ and NIC oversubscription ratio U_{NIC}, as shown in Lines 10-16 of Algorithm 1. When $U_{NIC} > 1$, either the network is not congested ($\psi < 1$), or rate inflation dominates network congestion ($U_{NIC} > \psi$). Thus, Odin proactively allocates NIC bandwidth among QPs by updating the calculated rate as

$$R_i \leftarrow R_i \cdot \left[\frac{\gamma}{U_{NIC}} + (1-\gamma)\right] \tag{5}$$

In this way, Odin proportionally scales down the aggregated calculated rate toward the NIC capacity. The allocation is executed asynchronously upon ACK reception, allowing U_{NIC} to gradually converge to 1.

Overall, by converting NIC competition into a virtual congestion signal, Odin ensures that the calculated rate is adjusted to the actual sending rate at the NIC, i.e., eliminating the rate inflation. Consequently, each congestion response can be executed effectively rather than being neutralized by NIC oversubscription. Moreover, proportional allocation preserves fairness among QPs sharing the same NIC. Combined with MIMD-based fast congestion response in the network, proactive bandwidth allocation enables Odin to react quickly and efficiently to congestion.

4.4 Ingress-Egress Bandwidth Matching

To be aware of the ingress-egress bandwidth mismatching, Odin systematically accounts for both the NIC oversubscription ratio and the network congestion degree. Based on the awareness, Odin performs ingress-egress matching by selectively compensating the AI factor. As shown in Fig. 6, when the NIC is oversubscribed, i.e., $U_{NIC} > 1$, and the network is not congested, i.e., $\psi < 1$, the absence of congestion indicates the presence of idle egress bandwidth. In this condition, Odin applies proactive NIC bandwidth allocation to eliminate rate inflation and simultaneously magnify the AI factor. Therefore, the QP can aggressively acquire the available egress bandwidth and seize NIC resources occupied by other QPs.

$$R_i \leftarrow R_i + \beta_{Init} \tag{6}$$

In contrast, when the NIC is not oversubscribed, i.e., $U_{NIC} < 1$, Odin also magnifies the AI factor to compensate all QPs at the NIC, driving the ingress bandwidth towards full utilization.

Overall, this compensation-based ingress-egress bandwidth matching mechanism allows Odin to indirectly align ingress injection with egress utilization without requiring switch-level traffic coordination. For QPs with inflated rates under non-congested networks, the additional AI factor helps them obtain idle network bandwidth and seize NIC resources occupied by other QPs. For QPs without rate inflation, the additional AI factor ensures that unused NIC capacity is promptly filled. Together, these behaviors enable Odin to keep both ingress and egress links highly utilized, thereby accelerating the completion time of A2A traffic. Note that this matching mechanism intentionally prioritizes completion time over short-term per-flow fairness.

4.5 Discussion

Why not treat NIC as a congestion point? The common multiple-bottleneck problem is addressed by existing CC algorithms through responding only to the most congested bottleneck [14, 17]. Although this solution enables accurate congestion elimination, it overlooks the idle bandwidth available at other hops along the path. As discussed in §3.3, the optimal completion of A2A traffic requires full utilization of all egress links throughout the path. Consequently, responding only to the most congested bottleneck may leave available bandwidth wasted, resulting in a longer completion time for A2A traffic. Besides, the ingress-egress bandwidth may not be limited at the NIC, but also may appear in the fabric under certain conditions. The ingress-egress matching among all network hops would be a future research topic.

Is Odin beneficial to classic data center workloads? Under classic workloads, the bottleneck is often located at the network, and the NIC would rarely be oversubscribed, so the proactive allocation does not work. Meanwhile, the compensation affects all QPs, which increases the value of β to $2*\beta_{init}$ and maintains good fairness. Moreover, the TBO suppression mechanism only works when both receiving TBO signals and $\psi < 1.2$, but the classic workloads rarely satisfy the two conditions simultaneously, making the TBO suppression mechanism inactive. Specifically, classic workloads typically involve only a few congested ports. If the congestion is severe enough to trigger TBO marks, the ψ will exceed 1.2. Conversely, if the congestion is light, i.e., $\psi < 1.2$, the TBO would be small. Overall,

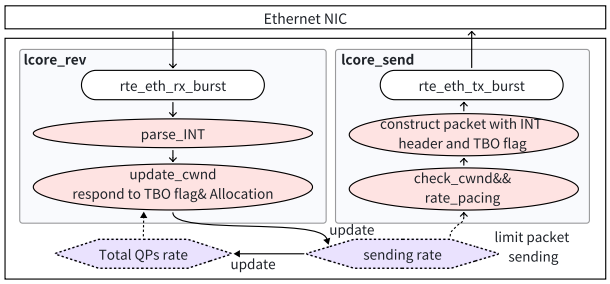


Figure 7: Framework of DPDK Implementation.

without these enhancement mechanisms, Odin degenerates into a special version of HPCC. Since HPCC has almost optimal performance for classic data center workloads, Odin naturally adapts well to such traffic.

5 Implementation

We implement Odin in DPDK [36] and implement the INT and TBO signals in the Tofino switch using P4 [37]. On the host side, all the key operations that a hardware RoCEv2 NIC would execute, including ACK/TBO feedback parsing, rate adjustment, and per-QP pacing, are implemented in DPDK as shown in Fig. 7. Specifically, data packets are constructed with a reserved space for INT metadata. The receiver copies the INT metadata into the ACK packet. When receiving an ACK packet, the sender parses the queue length, the number of bytes port sent, and the TBO flag of each hop. Based on the parsed information, the sending rate is calculated according to Algorithm 1. Correspondingly, the total QPs rate $\sum_i R_i$, which is recorded by only one variable in Odin, is updated. In addition to being updated after any QP receives an ACK packet, $\sum_i R_i$ is also updated when a QP is established, destroyed, or becomes inactive. Moreover, the sending of packets is controlled by both congestion window and pacing mechanisms in our DPDK prototype, the same as [14]. On the switch side, our implementation uses just one variable to count TBO, 1 arithmetic logic unit to check whether the TBO threshold has been reached, and appends the TBO flag and other telemetry information in the reserved INT metadata space of data packets.

Moreover, the overhead of Odin is acceptable. In detail, according to Algorithm 1, Odin additionally introduces 7 comparison/addition/subtraction(cas) operations, 3 multiplications(mul), and 1 division(div) compared to HPCC, which requires 25 cas, 9 mul, and 8 div operations under a two-layer Clos topology with 3 hops along the network path [14]. Assume 1 clock cycle for cas, 3 cycles for mul, and 16 cycles for div according to the public RISC-V core model [38], HPCC requires 25*1+9*3+8*16=180 cycles, while Odin adds only 7*1+3*3+1*16=32 cycles. This lightweight computation contributes a lot to the line-rate processing of our DPDK prototype. Furthermore, it also indicates that Odin is suitable for line-rate execution in programmable CC engines.

In addition, the implementation of Odin does not require an ASIC redesign or new physical hardware, but it does require lightweight support from programmable switch pipelines and programmable CC engines. First, all the above host operations can be customized in programmable CC engines such as NVIDIA BlueField-3 DPU [39] and ConnectX-8 SuperNIC [40]. Second, it is feasible to generate and propagate TBO signals in programmable switches. On the one hand, INT has been supported by commercial switches such as Broadcom [41] and NVIDIA Spectrum [42]. Odin can use IFA2.0 INT formats [43] to carry telemetry metadata. The 1-bit TBO signal can be encoded into the INT metadata using standard INT hardware pipelines, together with other states such as queue length or timestamps [44]. Other telemetry formats, such as IETF RFC 9197 IOAM [45], can also carry the TBO signal via implementation-defined fields. On the other hand, shared-buffer switches with dynamic-threshold PFC already maintain switch-level buffer accounting for buffer management. Although this switch-level buffer accounting value have not been directly exposed in current commodity switches, Odin may reuse it to generate TBO signals.

6 Evaluation

6.1 Setup

We built a testbed consisting of 8 Dell Precision 5820 tower workstations. Each machine is equipped with a Mellanox CX5 NIC, and all hosts are connected through a Tofino 1 switch. The PFC mechanism is enabled on the switch. We implement Odin, DCQCN [13], and HPCC [14] in DPDK, and disable the hardware DCQCN inside the NIC. The basic RTT is less than 10us. We also conduct large-scale NS3 [31] simulations to evaluate Odin. The simulator is based on the open-source code provided by [14]. We adapt the traffic-generation logic of SimAI [46] to emulate collective communication operations, including A2A and All-Reduce. The generator supports realistic MoE-based A2A traffic, NCCL-based asynchronous A2A traffic, and constructed synchronous A2A traffic. By default, 10 QPs are configured between each pair of nodes. The default topology is shown in Fig. 1(a). The link bandwidth is 400Gbps, the basic RTT is 8us, and the maximum transmission unit (MTU) is 4KB. The shared buffer size of the switch is configured to 82MB. The PFC threshold is dynamic according to [32]. Equal-Cost Multi-Path Routing is employed by default. More detailed configurations and discussions on their realism and representativeness are shown in Appendix C. Odin set α_{TBO} as 1/16 to maintain a low TBO, and set the EWMA factor γ as 0.1, according to the experiences in §6.3. We compare Odin with DCQCN, Timely, HPCC, PowerTCP, and Poseidon [17], using the same parameters as in their original papers. Poseidon iteratively adjusts sending rates to achieve a target port buffer occupancy similar to Timely, but uses INT similar to HPCC.

6.2 Testbed Experiment

In the testbed experiment, A2A traffic with 8 nodes is started concurrently. The maximum rate from the NIC to the switch is 40Gb/s, and the total maximum throughput of the switch is 320Gb/s. In this small testbed, the initial TBO generated by A2A traffic is relatively small according to the calculations in the appendix A. However, DCQCN still results in a large TBO of about 2MB as shown in Fig. 8 due to its spontaneous rate increase as explained in §2.2.2. Similarly, due to the small number of flows in the switch, the AI factor of HPCC will not result in a significant TBO. Although HPCC can maintain a near-zero TBO, it sacrifices significant throughput because it is extremely sensitive to small buffer occupancy of ports.

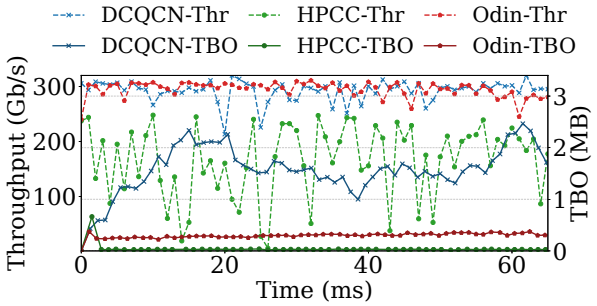


Figure 8: Testbed results.

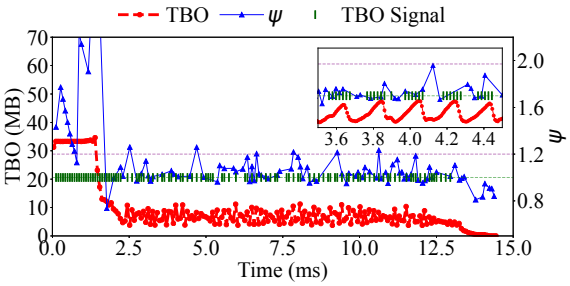


Figure 9: The TBO Suppression of Odin.

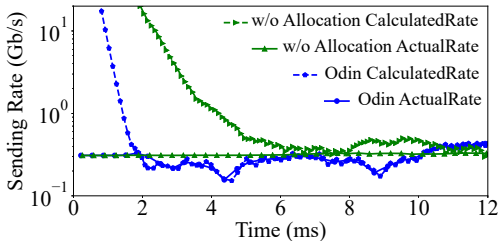


Figure 10: Odin avoids rate inflation. w/o Allocation denotes disabling Odin's proactive bandwidth allocation and compensation for bandwidth matching.

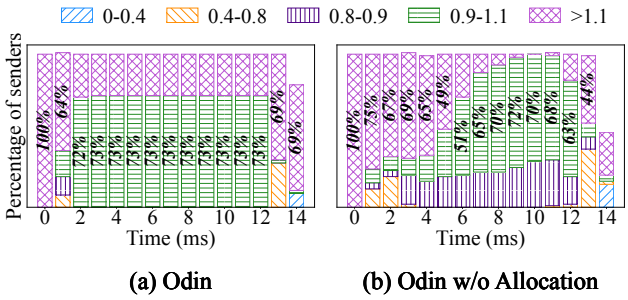


Figure 11: Ingress link utilization. Legend is the NIC oversubscription ratio interval.

In contrast, Odin can stably maintain small TBO and consistently achieve high throughput, reducing TBO by 78.7% over DCQCN and improving throughput by 1.75× compared to HPCC. On the one hand, the TBO suppression mechanism allows Odin to perceive changes in TBO and control the growth of TBO by responding to TBO signals. On the other hand, Odin eliminates the rate inflation by proactive bandwidth allocation, thereby ensuring valid execution of fast responsiveness. Moreover, Odin improves link utilization by its compensation for ingress-egress bandwidth matching.

6.3 Basic Properties

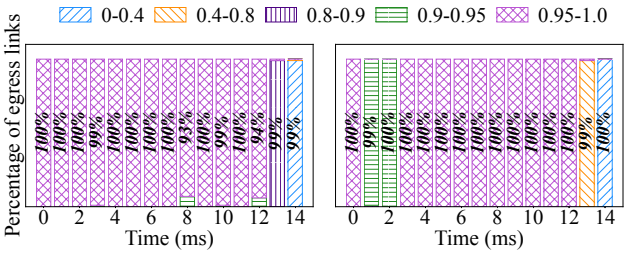
We evaluate Odin using the same scenario as Fig. 2 to show how Odin suppresses TBO, eliminates inflated rates, and matches ingress-egress bandwidth.

Odin suppresses TBO small. As shown in Fig. 9, during $0 \sim 2ms$, Odin does not respond to the TBO signal, while leveraging the MIMD rules to first suppress the port congestion. After $2ms$, Odin observes that the port congestion is relatively slight, i.e., $\psi < 1.2$, but the TBO is still large. Accordingly, Odin starts to respond to TBO signals. As shown in the sub-graph of Fig. 9, Odin effectively suppresses TBO by disabling the AI factor after receiving TBO signals (green marks). When the TBO is suppressed, the TBO signal is not generated and the AI factor is recovered again for good fairness. Therefore, the TBO slightly oscillates around the TBO threshold $82/16 = 5.125MB$ in Fig. 9.

Odin eliminates rate inflation. We present the calculated rate and actual rate of a sampled flow in Fig. 10. At the beginning, the calculated rate is the line rate of $400Gbps$, and the actual sending rate is $400Gbps/630 \approx 635Mbps$. In Odin w/o Allocation, the calculated rate is higher than the actual rate for a longer duration. In other words, the congestion response is ineffectual until $6ms$. But Odin quickly eliminates rate inflation by referring to the NIC oversubscription ratio and thus the calculated rate almost keeps aligned with the actual rate. Accordingly, Odin reduces the time to eliminate the inflated rate by more than 50%.

Odin matches ingress-egress bandwidth. The ingress link utilization can be implied by NIC oversubscription ratio. Therefore, we record the NIC oversubscription ratio within each 1ms time window and show percentage of senders with similar NIC oversubscription ratio, i.e., within the same interval in the legend of Figure 11. Odin senders always maintains over 90% ingress link utilization as shown in Fig. 11(a). This stems from that Odin proactively allocates bandwidth to eliminate rate inflation and compensates bandwidth for ingress ports experiencing utilization loss. In contrast, as shown in Fig. 11(b), Odin w/o Allocation suffers from severe rate inflation, misleading rate adjustment, and resulting in frequent ingress link utilization loss (<0.9).

Similarly, Fig. 12 illustrates the time-varying distribution of egress link utilization across all ports. As shown in Fig. 12(a), Odin maintains 95% egress utilization for almost all ports, because it quickly obtains idle egress bandwidth by compensating QP with additional rate. Conversely, Fig. 12(b) shows Odin w/o Allocation suffers from severe utilization loss. During $1 \sim 2ms$, most egress ports achieve only 90-95% utilization due to overreaction. During



(a) Odin (b) Odin w/o Allocation

Figure 12: Egress link utilization. Legend is the egress link utilization interval.

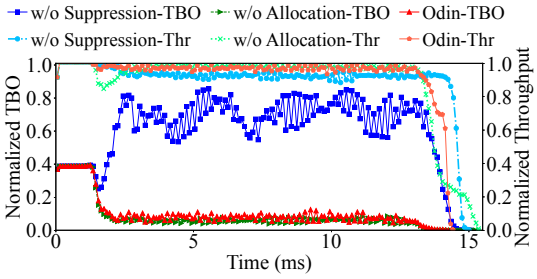


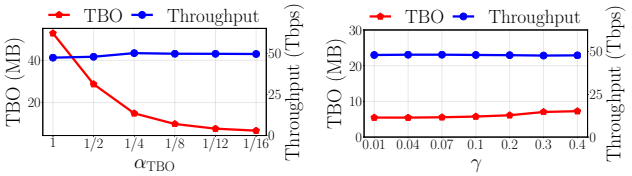
Figure 13: Ablation Test. w/o Suppression disables TBO suppression (§4.2), whereas w/o Allocation disables proactive bandwidth allocation (§4.3) and ingress-egress bandwidth matching (§4.4).

$13 \sim 14ms$ when many QPs complete, the remaining QPs cannot quickly obtain idle bandwidth and lose more than 60% throughput.

Overall, Odin closely matches the ingress-egress bandwidth, thereby accelerating the completion of A2A traffic.

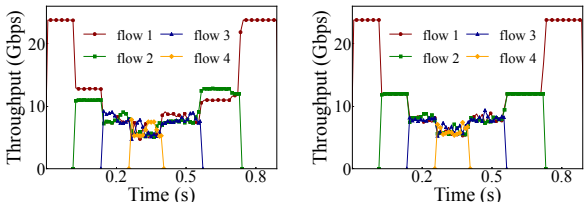
Ablation study. The results are shown in Fig. 13. On the one hand, Odin w/o Suppression has a large TBO, because its port buffer occupancy cannot be controlled to 0. This indicates the necessity of TBO signals. On the other hand, Odin w/o Allocation struggles to eliminate rate inflation and the responsiveness becomes ineffectual. Consequently, at 2ms, Odin w/o Allocation suffers from near 20% throughput loss. In contrast, Odin can acquire idle bandwidth quickly at 2ms. Moreover, after 14ms, some QPs still hold data to be transmitted due to prior ingress-egress bandwidth mismatch in Odin w/o Allocation. Accordingly, Odin w/o Allocation prolongs the task completion time of A2A traffic. In contrast, Odin avoids this issue via ingress-egress bandwidth matching.

Parameter Sensitivity. The values of α_{TBO} in Eq. (1) and the EWMA factor γ in Eq. (4) are changed, and the corresponding results are shown in Fig. 14. On the one hand, Fig. 14(a) shows that the TBO decreases along with the decrease of α_{TBO} from 1 to 1/16. This confirms the effectiveness of the TBO suppression mechanism. Meanwhile, the throughput remains almost unchanged. In total, Odin suppresses excessive TBO to avoid harmful buffer buildup, without damaging the throughput. On the other hand, Fig. 14(b) shows that TBO and throughput remain almost unchanged as γ



(a) Different α_{TBO} (b) Different γ

Figure 14: Parameter Sensitivity.



(a) HPCC (b) Odin

Figure 15: Fairness.

increases from 0.01 to 0.4. This is because γ mainly controls how fast the inflated aggregated rate is reduced below the NIC capacity, i.e., the rate-inflation elimination speed. However, severe rate inflation in Odin usually appears during the flow startup phase (0-2ms in Fig. 10) and becomes mild thereafter (2-12ms in Fig. 10). Therefore, the inflation-elimination speed controlled by γ has a limited impact on both TBO and throughput of Odin.

Odin achieves good fairness. Following the fairness experiment in HPCC [14], we let 4 flows join a link one by one every 0.1 second and leave afterwards. As shown in Fig. 15, Odin achieves comparable fairness to HPCC. This is because Odin would be similar to HPCC under non-A2A traffic, as discussed in §4.5.

6.4 Performance Comparison

We compare Odin with existing CC algorithms under A2A traffic, prefill-decode disaggregation traffic, and workloads combined by both A2A and All-Reduce traffic.

Odin achieves the lowest TBO and the highest throughput under large-scale A2A traffic. A2A traffic with a scale of 512 nodes is generated in a two-layer leaf-spine topology with convergence ratio of 1:1. Here, 64 nodes are connected to each leaf switch with link rate of 400Gb/s. As shown in Fig. 16, DCQCN, Timely, Poseidon, and PowerTCP all suffer from large TBO. Although the TBO of HPCC is relatively small, it is still larger than Odin, and HPCC significantly loses throughput. Compared to DCQCN, Timely, HPCC, PowerTCP, and Poseidon, Odin reduces the TBO by 88%, 78%, 55%, 80%, and 76%, respectively.

Odin reduces task completion time (TCT) under asynchronous A2A traffic. According to NCCL [47], the asynchronous A2A traffic is initiated in batches of up to 8 QPs, with random initiation intervals of up to $50\mu s$. Table 2 shows that Odin achieves the best TCT of A2A traffic without triggering PFC. Specifically, Odin reduces TCT by 42.24%, 71.62%, 20.78%, 5.18%, and 17.8% compared

Table 2: Asynchronous A2A traffic.

CC	Thr(Gb/s)	Pause	TCT(ms)
DCQCN	160	2,525	3.18
Timely	78	2,537	6.45
HPCC	219	0	2.31
PowerTCP	263	0	1.93
Poseidon	227	0	2.23
Odin	**276**	0	**1.83**

Table 3: Prefill-Decode scenario.

CC	Thr(Gb/s)	Pause	TCT(ms)
DCQCN	309.46	72,678	8.34
Timely	155.72	49,356	12.74
HPCC	341.63	**2,096**	6.16
PowerTCP	360.41	4,884	5.83
Poseidon	354.93	2,852	5.92
Odin	**378.62**	**2,098**	**5.56**

Table 4: All-Reduce scenario.

CC	Pause	TCT(ms)
DCQCN	91,817	1575.67
Timely	25,890	6060.21
HPCC	0	768.16
PowerTCP	0	498.70
Poseidon	0	501.61
Odin	0	**446.10**

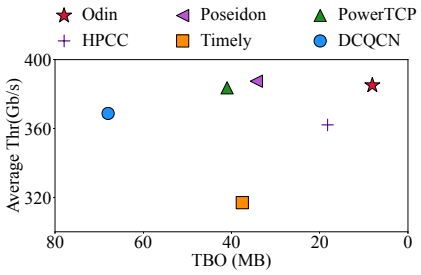


Figure 16: TBO and throughput under A2A traffic.

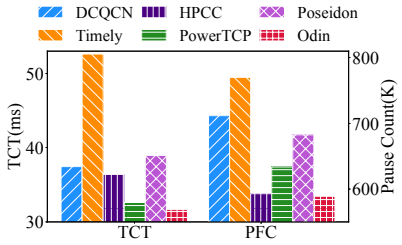


Figure 17: Realistic workload with MoE model.

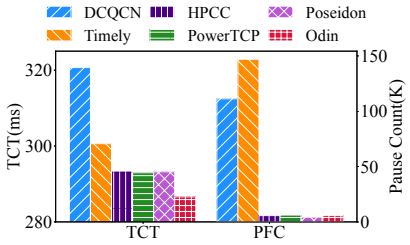


Figure 18: Realistic workload with AlexNet model.

to DCQCN, Timely, HPCC, PowerTCP, and Poseidon, respectively. This is because many QPs finished earlier due to the asynchronous initialization and existing CCs cannot match ingress-egress bandwidth due to Feature 1. Accordingly, the throughputs of existing CCs are lower than Odin. Moreover, the three counterintuitive phenomena observed in § 2.2.1 persist under asynchronous starting times within (0,1) μs as shown in Appendix D.2.

Odin has best performance under non-A2A traffic such as prefill-decode disaggregation. The ratio of prefill nodes to decode nodes is set to 3:1, the same as [48]. Each prefill node transmits 2.5MB of tokens. The results are shown in Table 3. Odin achieves 20Gb/s higher throughput, has nearly the smallest PFC pauses, and reduces TCT by over 5% than other INT-based CC algorithms. Compared to DCQCN and Timely, Odin reduces TCT by 33% and 56.4%, respectively. This is because DCQCN and Timely suffer from sparse rate adjustment and rate inflation due to the large number of QPs. In contrast, Odin ensures valid execution of its fast rate adjustment through proactive bandwidth allocation.

Odin performs best under the combination of A2A and All-Reduce traffic. We construct the scenario including 144 nodes, with 128 running A2A traffic, 12 running All-Reduce traffic, and 4 carrying east–west short-message traffic. In this scenario, A2A traffic generates large TBO and may incur PFC pauses, which would delay the completion of co-located All-Reduce traffic in the same cluster. As shown in Table 4, Odin respectively reduces the TCT of the All-Reduce traffic by 72% and 93% compared to DCQCN and Timely, because DCQCN and Timely cannot suppress TBO. Compared to HPCC, PowerTCP, and Poseidon, Odin reduces the TCT by 41.92%, 10.55%, and 11.07%, respectively. In addition, Odin also achieves good performance under the pure all-reduce traffic in Appendix D.3.

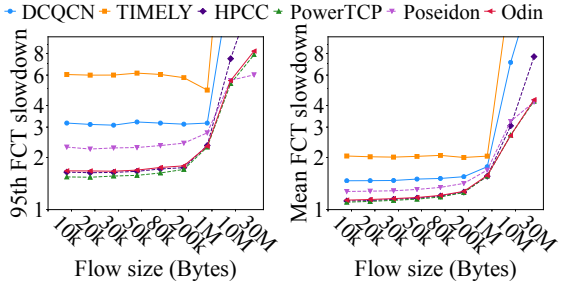


Figure 19: FCT slowdown, i.e., actual FCT/ideal FCT, over classic datacenter workloads.

6.5 Realistic Workloads

We test Odin with realistic workloads, including the MoE model training traces [46] where A2A traffic takes the majority, as well as the non-A2A workloads, including the AlexNet traces [18] and the classic data center workload [14].

Odin reduces the frequency of triggering PFC and TCT in the MoE model training. We use SimAI [46] to generate traces of Spectrum-X 400Gbps topology and *A100_Mixtral_MoE* model with a scale of 128 GPUs. As shown in Fig. 17, Odin reduces the number of PFC pauses by 18%, 24%, 8%, and 14% compared with DCQCN, Timely, PowerTCP, and Poseidon, respectively. Although HPCC has a small number of PFC pauses like Odin, it prolongs TCT due to the ingress-egress bandwidth mismatch. Compared with DCQCN, Timely, HPCC, PowerTCP, and Poseidon, Odin reduces the TCT of MoE model training by 16%, 40%, 13%, 3%, and 19%, respectively. PowerTCP achieves a similar TCT as Odin, because it holds a relatively large port buffer occupancy and would not lose

throughput. However, it triggers more PFC pauses than Odin due to the large TBO.

Odin achieves similar performance as existing INT-based CC algorithms under non-A2A traffic. Firstly, we use the AlexNet [18] training workload, which comprises multiple rounds of forward and back propagation. The same as [49], a leaf-spine topology, consisting of 128 training nodes and 4 parameter server nodes, is applied. As shown in Fig. 18, Odin achieves similar TCT and PFC pauses with existing INT-based CC algorithms, because the rate adjustment rules of Odin become similar to HPCC under non-A2A scenarios. Similarly, as shown in Fig. 19, under the classic web search traffic [14] in data centers, Odin still achieves almost the best 95th percentile and mean flow completion time (FCT) similar to existing INT-based CC algorithms.

7 Related Work

A2A Communication. Recently, A2A traffic transmission is optimized in terms of collective communication libraries by works such as TACCL [50] and TECCL [51], scheduling strategies and resource allocation by works such as Lina [6] and Janus [5], and architecture by works such as Parm [52] and DeepSpeed-MoE [7]. In contrast, Odin focuses on the transport-level transmission acceleration and is orthogonal to these efforts.

Multi-path Transmission and Load Balance (LB). Recent works such as UET [53], Falcon [54], Ethereal [55], REPS [56], SMaRTT [57], and MRC/SRv6 [58], mitigate load imbalance and improve path utilization through packet spraying, congestion-aware path selection, application-level flow split, or explicit source routing. Under A2A traffic in a Clos topology, all ToR downlinks are saturated simultaneously, leaving no under-utilized paths for LB to exploit. Thus, LB hardly takes effect under A2A traffic. Odin is orthogonal to these works, since it addresses A2A-induced issues at the transport layer after traffic has been placed on paths by LB. Although certain CC functions are employed by works [53, 54, 57], they are in the traditional design space of CC discussed in §3. Odin is complementary to them because Odin serves as an enhancement to the existing CC by being aware of TBO and forcing CC to operate on actual sending rates rather than inflated rates.

Congestion Control in Datacenter. Existing CC algorithms, including DCQCN [13], Timely [15], HPCC [14], and PowerTCP [16], cannot fulfill the requirements of A2A traffic, as shown in Table 1. Poseidon [17] also has a large TBO in Fig. 16. Similarly, recent works [59–62] are rarely aware of the issues under A2A traffic. Improvement similar to Odin may help to enhance their performance under A2A traffic.

Receiver-driven CC algorithms, such as pHost [63], ExpressPass [64], NDP [65], Homa [66], Aeolus [67], Fork [68], and SIRD [34] allocate credits according to the perceived congestion status. While they may improve A2A transmission, their deployment is challenging due to requirements for specialized switch support and significant switch modification overhead. Conversely, Odin is simple and easy to deploy.

Learning-based CC algorithms [69–73] may work well under A2A traffic, but they are hard for humans to reason about and need an expensive training process. In contrast, Odin is interpretable with acceptable overhead.

Coordination of NIC and CC. Many scheduling algorithms on NIC, such as WF^2Q^+ [74], SENIC [75], Carousel [76], and Tassel [77], are designed for setting accurate actual sending rates for each QP according to their calculated rates. They just ensure proportional rate allocation when the NIC is oversubscribed, and the rate may still be inflated.

Buffer Management and Active Queue Management. Buffer management schemes, such as DT [32], EDT [78], ABM [79], and Reverie [80], primarily focus on allocating the switch buffer for each port, while active queue management schemes, such as RED [81], CoDel [82], ACC [70], and PRED [26], focus on the suppression of port buffer occpuancy. They are orthogonal to Odin, which suppresses TBO with the help of large TBO signals at switches. Moreover, Meta [10] allows large TBO in leaf switches and uses deep-buffer spine switches to prevent PFC from propagating further, while Odin fundamentally eliminates the large TBO.

8 Conclusion

This paper reveals that A2A traffic causes large TBO and performance degradation. The underlying reasons are the bandwidth saturation of all ports, the rate inflation of QP itself, and the interference among QPs at the NIC. These motivate us to rethink the design space of CC under A2A traffic and show that several traditional design methods adopted by existing CC algorithms are ineffective. Aware that A2A traffic narrows the design space of CC, we design Odin. Odin is a systematic external enhancement to modern CC, as it systematically resolves new issues caused by A2A traffic. (1) TBO Suppression addresses the excessive accumulation of shared buffer, as the TBO threshold is a deterministic function of switch buffer size. (2) Proactive Allocation mathematically synchronizes the theoretical CC rate with the NIC's physical limits. (3) Ingress-Egress Bandwidth matching resolves uncongested-network/congested-NIC discrepancies. Consequently, these mechanisms remain applicable across deployments where A2A traffic creates switch-level TBO pressure, NIC oversubscription, and ingress-egress bandwidth mismatching. Testbed experiments and extensive simulations, including the realistic MoE model training workload, AlexNet training workload, prefill-decode disaggregation traffic, and classic datacenter workloads, show that Odin achieves low TBO and high throughput under A2A traffic, while keeping strong performance under non-A2A traffic.

Ethics: This work does not raise any ethical issues.

Acknowledgment

We gratefully acknowledge the anonymous reviewers for their constructive comments and suggestions. We especially thank Haoyang Li and Yuxuan Hu for their valuable technical discussions and insightful contributions throughout this project. This work is supported in part by the National Natural Science Foundation of China under Grant No. 62472451 and the Science and Technology Innovation Program of Hunan Province under Grant No. 2023RC3047. In addition, this work was also carried out in part using computing resources at the High Performance Computing Center of Central South University.

References

[1] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P. Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. GLaM: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 5547–5569, Baltimore, MD, USA, July 2022.

[2] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. https://arxiv.org/abs/1701.06538, 2017. Accessed: 2026-07-03.

[3] Dianhai Yu, Liang Shen, Hongxiang Hao, Weibao Gong, Huachao Wu, Jiang Bian, Lirong Dai, and Haoyi Xiong. MoESys: A Distributed and Efficient Mixture-of-Experts Training and Inference System for Internet Services. https://arxiv.org/abs/2205.10034, 2024. Accessed: 2026-07-03.

[4] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, and Wenfeng Liang. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, September 2025.

[5] Juncai Liu, Jessie Hui Wang, and Yimin Jiang. Janus: A unified distributed training framework for sparse mixture-of-experts models. In *Proceedings of the ACM SIGCOMM 2023 Conference*, SIGCOMM '23, pages 486–498, New York, NY, USA, September 2023.

[6] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed MoE training and inference with Lina. In *Proceedings of the 2023 USENIX Annual Technical Conference*, ATC '23, pages 945–959, Boston, MA, USA, July 2023.

[7] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *ICML '22*, pages 18332–18346, Baltimore, MD, USA, July 2022.

[8] Yanpei Chen, Rean Griffith, Junda Liu, Anthony D. Joseph, and Randy H. Katz. Understanding TCP incast throughput collapse in datacenter networks. In *Proceedings of the 1st ACM Workshop on Research on Enterprise Networking*, WREN '09, pages 73–82, Barcelona, Spain, August 2009.

[9] Hongsik Choi, Suresh Subramaniam, and Hyeong-Ah Choi. Optimal Wavelength Assignment Algorithms for Permutation Traffic in Multi-Fiber WDM Ring Networks. *Photonic Network Communications*, 4(1):37–46, January 2002.

[10] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. Rdma over ethernet for distributed training at meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, SIGCOMM '24, pages 1–14, Sydney, NSW, Australia, August 2024.

[11] IEEE. 802.1 qbb—priority-based flow control. IEEE Standard IEEE Std 802.1Qbb-2011, Institute of Electrical and Electronics Engineers, 2008.

[12] InfiniBand Trade Association. Infiniband Architecture Specification Volume 2 Release 1.2. https://www.cs.kent.edu/~farrell/cc05/reference/infiniband/specs/vol2r1_2.pdf, 2004. Accessed: 2026-07-03.

[13] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for large-scale RDMA deployments. In *Proceedings of the ACM SIGCOMM 2015 Conference*, SIGCOMM '15, pages 523–536, London, United Kingdom, August 2015.

[14] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. HPCC: High precision congestion control. In *Proceedings of the ACM SIGCOMM 2019 Conference*, SIGCOMM '19, pages 44–58, Beijing, China, August 2019.

[15] Radhika Mittal, Vinh The Lam, Nandita Dukkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. TIMELY: RTT-based congestion control for the datacenter. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, SIGCOMM '15, pages 537–550, New York, NY, USA, August 2015.

[16] Vamsi Addanki, Oliver Michel, and Stefan Schmid. PowerTCP: Pushing the performance limits of datacenter networks. In *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation*, NSDI '22, pages 51–70, Renton, WA, April 2022.

[17] Weitao Wang, Masoud Moshref, Yuliang Li, Gautam Kumar, T. S. Eugene Ng, Neal Cardwell, and Nandita Dukkipati. Poseidon: Efficient, robust, and practical datacenter CC via deployable INT. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation*, NSDI '23, pages 255–274, Boston, MA, USA, April 2023.

[18] Alex Krizhevsky, Ilya Sutskever, and Geoffrey Everest Hinton. ImageNet Classification with Deep Convolutional Neural Networks. *Communications of the ACM*, 60(6):84–90, June 2017.

[19] Emmanuel Bengio, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. Conditional Computation in Neural Networks for faster models. https://arxiv.org/abs/1511.06297, 2016. Accessed: 2026-07-03.

[20] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. https://arxiv.org/abs/1308.3432, 2013. Accessed: 2026-07-03.

[21] Jan Ludziejewski, Maciej Pióro, Jakub Krajewski, Maciej Stefaniak, Michał Krutul, Jan Małaśnicki, Marek Cygan, Piotr Sankowski, Kamil Adamczewski, Piotr Miłoś, and Sebastian Jaszczur. Joint MoE Scaling Laws: Mixture of Experts Can Be Memory Efficient. https://arxiv.org/abs/2502.05172, 2025. Accessed: 2026-07-03.

[22] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. https://arxiv.org/abs/2101.03961, 2022. Accessed: 2026-07-03.

[23] Young Jin Kim, Ammar Ahmad Awan, Alexandre Muzio, Andres Felipe Cruz Salinas, Liyang Lu, Amr Hendy, Samyam Rajbhandari, Yuxiong He, and Hany Hassan Awadalla. Scalable and Efficient MoE Training for Multitask Multilingual Models. https://arxiv.org/abs/2109.10465, 2021. Accessed: 2026-07-03.

[24] Steven James Plimpton. Fast Parallel Algorithms for Short-Range Molecular Dynamics. *Journal of Computational Physics*, 117(1):1–19, March 1995.

[25] Berk Hess, Carsten Kutzner, David van der Spoel, and Erik Lindahl. GROMACS 4: Algorithms for Highly Efficient, Load-Balanced, and Scalable Molecular Simulation. *Journal of Chemical Theory and Computation*, 4(3):435–447, March 2008.

[26] Xinle Du, Tong Li, Guangmeng Zhou, Zhuotao Liu, Hanlin Huang, Jiaming Zhang, Zhaohui Zhang, and Ke Xu. PRED: Performance-oriented random early detection for consistently stable performance in datacenters. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation*, NSDI '25, pages 1019–1034, Philadelphia, PA, USA, April 2025.

[27] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitendra Padhye, and Marina Lipshteyn. RDMA over commodity ethernet at scale. In *Proceedings of the 2016 ACM SIGCOMM Conference*, SIGCOMM '16, pages 202–215, Florianopolis, Brazil, August 2016.

[28] Shuihai Hu, Yibo Zhu, Peng Cheng, Chuanxiong Guo, Kun Tan, Jitendra Padhye, and Kai Chen. Deadlocks in datacenter networks: Why do they form, and how to avoid them. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, HotNets '16, pages 92–98, Atlanta, GA, USA, November 2016.

[29] Shuihai Hu, Yibo Zhu, Peng Cheng, Chuanxiong Guo, Kun Tan, Jitendra Padhye, and Kai Chen. Tagger: Practical pfc deadlock prevention in data center networks. In *Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies*, CoNEXT '17, pages 313–326, Incheon, Republic of Korea, December 2017.

[30] Yixiao Gao, Qiang Li, Lingbo Tang, Yongqing Xi, Pengcheng Zhang, Wenwen Peng, Bo Li, Yaohui Wu, Shaozong Liu, Lei Yan, Fei Feng, Yan Zhuang, Fan Liu, Pan Liu, Xingkui Liu, Zhongjie Wu, Junping Wu, Zheng Cao, Chen Tian, Jinbo Wu, Jiaji Zhu, Haiyong Wang, Dennis Cai, and Jiesheng Wu. When cloud storage meets RDMA. In *18th USENIX Symposium on Networked Systems Design and Implementation*, NSDI '21, pages 519–533, Boston, MA, USA, April 2021.

[31] The ns-3 Project. ns-3: Network simulator. https://www.nsnam.org/. Accessed: 2026-07-03.

[32] Abhijit Kumar Choudhury and Ellen Lynn Hahne. Dynamic Queue Length Thresholds for Shared-Memory Packet Switches. *IEEE/ACM Transactions on Networking*, 6(2):130–140, April 1998.

[33] Wanchun Jiang, Yujie Hu, Haoyang Li, Kai Wang, Jiawei Huang, and Jianxin Wang. Analysis and improvement of PowerTCP. In *2024 IEEE/ACM 32nd International Symposium on Quality of Service*, IWQoS '24, pages 1–6, Hong Kong, China, June 2024.

[34] Konstantinos Prasopoulos, Ryan Kosta, Edouard Bugnion, and Marios Kogias. SIRD: A sender-informed, receiver-driven datacenter transport protocol. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation*, NSDI '25, pages 451–471, Philadelphia, PA, USA, April 2025.

[35] Philip Hall. On Representatives of Subsets. *Journal of the London Mathematical Society*, s1-10(1):26–30, January 1935.

[36] Linux Foundation. Data plane development kit (DPDK). http://www.dpdk.org. Accessed: 2026-07-03.

[37] P4 Language Consortium. p4lang/p4-spec: P4 programming language specification repository. https://github.com/p4lang/p4-spec. Accessed: 2026-07-03.

[38] RT-RK. Overview of SiFive 7 Series Microarchitecture in RISC-V GCC. https://www.rt-rk.com/overview-of-sifive-7-series-microarchitecture-in-risc-v-gcc/, 2024. Accessed: 2026-07-03.

[39] NVIDIA Corporation. DOCA-PCC: Doca programmable congestion control documentation. https://docs.nvidia.com/doca/sdk/DOCA-PCC/index.html, 2026. Accessed: 2026-07-03.

[40] NVIDIA Corporation. NVIDIA ConnectX-8 SuperNIC Datasheet. https://resources.nvidia.com/en-us-accelerated-networking-resource-library/connectx-datasheet, 2025. Accessed: 2026-07-03.

[41] Broadcom Inc. In-band Telemetry Product Brief. https://docs.broadcom.com/doc/IBT-PB100, June 2017. Accessed: 2026-07-03.

[42] NVIDIA Corporation. NVIDIA Spectrum SN4000 Open Ethernet Switches. https://www.nvidia.com/en-in/networking/ethernet-switching/spectrum-sn4000/, 2026. Accessed: 2026-07-03.

[43] Jai Kumar, Surendra Anubolu, John Lemon, Rajeev Manur, Hugh Holbrook, Anoop Ghanwani, Dezhong Cai, Heidi Ou, Yizhou Li, and Xiaojun Wang. Inband Flow Analyzer. Internet-Draft draft-kumar-ippm-ifa-02, Internet Engineering Task Force, April 2020.

[44] The P4.org Applications Working Group. In-band Network Telemetry (INT) Dataplane Specification v2.1. https://p4.org/wp-content/uploads/sites/53/p4-spec/docs/INT_v2_1.pdf, November 2020. Accessed: 2026-07-03.

[45] Frank Brockners, Shwetha Bhandari, and Tal Mizrahi. Data Fields for In Situ Operations, Administration, and Maintenance (IOAM). https://www.rfc-editor.org/info/rfc9197, May 2022. Accessed: 2026-07-03.

[46] Xizheng Wang, Qingxu Li, Yichi Xu, Gang Lu, Dan Li, Li Chen, Heyang Zhou, Linkang Zheng, Sen Zhang, Yikai Zhu, Yang Liu, Pengcheng Zhang, Kun Qian, Kunling He, Jiaqi Gao, Ennan Zhai, Dennis Cai, and Binzhang Fu. SimAI: Unifying architecture design and performance tuning for Large-Scale large language model training with scalability and precision. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation*, NSDI '25, pages 541–558, Philadelphia, PA, USA, April 2025.

[47] David Bruce Yoffie and Sarah von Bargen. Nvidia, Inc. in 2024 and the Future of AI. https://store.hbr.org/product/nvidia-inc-in-2024-and-the-future-of-ai/725360, September 2024. Accessed: 2026-07-03.

[48] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *ACM Transactions on Storage*, 22, March 2026.

[49] Wanchun Jiang, Haoyang Li, Kai Wang, Yujie Hu, Xiao Han, Danfeng Shan, Fengyuan Ren, Jiawei Huang, and Jianxin Wang. CCC: Re-architecting delay-based congestion control in datacenter networks. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation*, NSDI '26, pages 1007–1022, Renton, WA, USA, May 2026.

[50] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. TACCL: Guiding collective algorithm synthesis using communication sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation*, NSDI '23, pages 593–612, Boston, MA, USA, April 2023.

[51] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. Rethinking machine learning collective communication as a multi-commodity flow problem. In *Proceedings of the ACM SIGCOMM 2024 Conference*, SIGCOMM '24, pages 1–22, Sydney, NSW, Australia, August 2024.

[52] Xinglin Pan, Wenxiang Lin, Shaohuai Shi, Xiaowen Chu, Weinong Sun, and Bo Li. Parm: Efficient training of large sparsely-activated models with dedicated schedules. In *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, INFOCOM '24, pages 1880–1889, Vancouver, BC, Canada, May 2024.

[53] Ultra Ethernet Consortium. Ultra Ethernet Specification v1.0.1. Industry Specification UE-Specification-1.0.1, Ultra Ethernet Consortium, September 2025.

[54] Arjun Singhvi, Nandita Dukkipati, Prashant Chandra, Hassan M. G. Wassel, Naveen Kr. Sharma, Anthony Rebello, Henry Schuh, Praveen Kumar, Behnam Montazeri, Neelesh Bansod, Sarin Thomas, Inho Cho, Hyojeong Lee Seibert, Baijun Wu, Rui Yang, Yuliang Li, Kai Huang, Qianwen Yin, Abhishek Agarwal, Srinivas Vaduvatha, Weihuang Wang, Masoud Moshref, Tao Ji, David Wetherall, and Amin Vahdat. Falcon: A reliable, low latency hardware transport. In *Proceedings of the ACM SIGCOMM 2025 Conference*, SIGCOMM '25, pages 1–16, Coimbra, Portugal, September 2025.

[55] Vamsi Addanki, Prateesh Goyal, Ilias Marinos, and Stefan Schmid. Ethereal: Divide and conquer network load balancing in large-scale distributed training. https://arxiv.org/abs/2407.00550, July 2024. Accessed: 2026-07-03.

[56] Tommaso Bonato, Abdul Kabbani, Ahmad Ghalayini, Michael Papamichael, Mohammad Dohadwala, Lukas Gianinazzi, Mikhail Khalilov, Elias Achermann, Daniele De Sensi, and Torsten Hoefler. REPS: Recycled entropy packet spraying for adaptive load balancing and failure mitigation. In *Proceedings of the 21st European Conference on Computer Systems*, EuroSys '26, pages 367–382, Edinburgh, Scotland, UK, April 2026.

[57] Tommaso Bonato, Abdul Kabbani, Daniele De Sensi, Rong Pan, Yanfang Le, Costin Raiciu, Mark Handley, Mihai Brodschi, Timo Schneider, Nils Blach, Ahmad Ghalayini, Anup Agarwal, Daniel Santos Ferreira Alves, Michael Papamichael, and Torsten Hoefler. SMaRTT: Sender-based Marked Rapidly-adapting Trimmed & Timed Transport. https://arxiv.org/abs/2404.01630, April 2024. Accessed: 2026-07-03.

[58] Joao Araujo, Alex Chow, Mark Handley, Ryder Lewis, Christoph Paasch, Jitendra Padhye, Michael Papamichael, Greg Steinbrecher, Amin Tootoonchian, Lihua Yuan, Sreevatsa Anantharamu, Abhishek Dosi, Mohit Garg, Mahdieh Ghazimirsaeed, Torsten Hoefler, Deepal Jayasinghe, Jithin Jose, Abdul Kabbani, Guohan Lu, Yang Wang, K. Doddapaneni, Murali Garimella, Vipin Jain, Yanfang Le, H. Nagulapalli, S. Narayanan, Rong Pan, Rathina Sabesan, Raghava Sivaramu, Rip Sohan, Eric Davis, Dragos Dumitrescu, Mohan Kalkunte, Bhaswar Mitra, Guglielmo Morandin, Adrian Popa, Costin Raiciu, Eric Spada, John Spillane, Niranjan Vaidya, Aviv Barnea, Idan Burstein, Elazar Cohen, Yamin Friedman, Noam Katz, Masoud Moshref, Yuval Shpigelman, Shahaf Shuler, Shy Shyman, and Sayantan Sur. Resilient AI Supercomputer Networking Using MRC and SRv6. https://arxiv.org/abs/2605.04333, May 2026. Accessed: 2026-07-03.

[59] Gautam Kumar, Nandita Dukkipati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. Swift: Delay is simple and effective for congestion control in the datacenter. In *Proceedings of the ACM SIGCOMM 2020 Conference*, SIGCOMM '20, pages 309–322, Virtual Event, USA, August 2020.

[60] Yajuan Peng, Haoran Wei, Xiaolong Zhong, Junkai Huang, Haohan Xu, Zicheng Wang, Yang Bai, Zhuo Jiang, Jianxi Ye, Xiaoliang Wang, Xiaoming Fu, and Huichen Dai. Barre: empowering simplified and versatile programmable congestion control in high-speed ai clusters. In *2025 USENIX Annual Technical Conference*, ATC '25, pages 1041–1056, Boston, MA, USA, July 2025.

[61] Zhaochen Zhang, Feiyang Xue, Rui Ning, Keqiang He, Gianni Antichi, Jiaqi Gao, Zhimeng Yin, Kexin Liu, Rui Li, Zhengqi Cui, Zhehao Lin, Peirui Cao, Guihai Chen, and Chen Tian. OSCAR: O(1)-Step convergence and readily-deployable congestion control. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation*, NSDI '26, pages 543–570, Renton, WA, May 2026.

[62] Wanchun Jiang, Haoyang Li, Kai Wang, Yujie Hu, Xiao Han, Danfeng Shan, Fengyuan Ren, Jiawei Huang, and Jianxin Wang. CCC: Re-architecting delay-based congestion control in datacenter networks. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation*, NSDI '26, pages 1553–1570, Renton, WA, May 2026.

[63] Peter Xiang Gao, Akshay Narayan, Gautam Kumar, Rachit Agarwal, Sylvia Ratnasamy, and Scott Shenker. pHost: distributed near-optimal datacenter transport over commodity network fabric. In *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies*, CoNEXT '15, pages 1–12, Heidelberg, Germany, December 2015.

[64] Inho Cho, Keon Jang, and Dongsu Han. ExpressPass: Credit-scheduled delay-bounded congestion control for datacenters. In *Proceedings of the ACM SIGCOMM 2017 Conference*, SIGCOMM '17, pages 239–252, Los Angeles, CA, USA, August 2017.

[65] Mark Handley, Costin Raiciu, Alexandru Agache, Andrei Voinescu, Andrew W. Moore, Gianni Antichi, and Marcin Wójcik. NDP: Re-architecting datacenter networks and stacks for low latency and high performance. In *Proceedings of the ACM SIGCOMM 2017 Conference*, SIGCOMM '17, pages 29–42, Los Angeles, CA, USA, August 2017.

[66] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. Homa: a receiver-driven low-latency transport protocol using network priorities. In *Proceedings of the ACM SIGCOMM 2018 Conference*, SIGCOMM '18, pages 221–235, Budapest, Hungary, August 2018.

[67] Shuihai Hu, Wei Bai, Gaoxiong Zeng, Zilong Wang, Baochen Qiao, Kai Chen, Kun Tan, and Yi Wang. Aeolus: A building block for proactive transport in datacenters. In *Proceedings of the ACM SIGCOMM 2020 Conference*, SIGCOMM '20, pages 422–434, Virtual Event, USA, July 2020.

[68] Yuan Liu, Wenxin Li, Yulong Li, Lide Suo, Xuan Gao, Xin Xie, Sheng Chen, Ziqi Fan, Wenyu Qu, and Guyue Liu. Fork: A dual congestion control loop for small and large flows in datacenters. In *Proceedings of the 20th European Conference on Computer Systems*, EuroSys '25, pages 402–417, Leiden, Netherlands, March 2025.

[69] Chen Tessler, Yuval Shpigelman, Gal Dalal, Amit Mandelbaum, Doron Haritan Kazakov, Benjamin Fuhrer, Gal Chechik, and Shie Mannor. Reinforcement Learning for Datacenter Congestion Control. *SIGMETRICS Performance Evaluation Review*, 49(2):43–46, January 2022.

[70] Siyu Yan, Xiaoliang Wang, Xiaolong Zheng, Yinben Xia, Derui Liu, and Weishan Deng. ACC: automatic ecn tuning for high-speed datacenter networks. In *Proceedings of the ACM SIGCOMM 2021 Conference*, SIGCOMM '21, pages 384–397, Virtual Event, USA, August 2021.

[71] Salma Emara, Fei Wang, Baochun Li, and Timothy Zeyl. Pareto: Fair Congestion Control With Online Reinforcement Learning. *IEEE Transactions on Network Science and Engineering*, 9(5):3731–3748, October 2022.

[72] Xudong Liao, Han Tian, Chaoliang Zeng, Xinchen Wan, and Kai Chen. Astraea: Towards fair and efficient learning-based congestion control. In *Proceedings of the Nineteenth European Conference on Computer Systems*, EuroSys '24, pages 1–16, Athens, Greece, April 2024.

[73] Han Tian, Xudong Liao, Decang Sun, Chaoliang Zeng, Yilun Jin, Junxue Zhang, Xinchen Wan, Zilong Wang, Yong Wang, and Kai Chen. Jury: Achieving fairness generalizability for learning-based congestion control. In *Proceedings of the 20th European Conference on Computer Systems*, EuroSys '25, pages 413–427, Leiden, Netherlands, March 2025.

[74] Jon Charles Robert Bennett and Hui Zhang. Hierarchical Packet Fair Queueing Algorithms. *IEEE/ACM Transactions on Networking*, 5(5):675–689, October 1997.

[75] Sivasankar Radhakrishnan, Yilong Geng, Vimalkumar Jeyakumar, Abdul Kabbani, George Porter, and Amin Vahdat. SENIC: scalable nic for end-host rate limiting. In *Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation*, NSDI '14, pages 409–422, Seattle, WA, USA, April 2014.
[76] Ahmed Saeed, Nandita Dukkipati, Vytautas Valancius, Vinh The Lam, Carlo Contavalli, and Amin Vahdat. Carousel: Scalable traffic shaping at end hosts. In *Proceedings of the ACM SIGCOMM 2017 Conference*, SIGCOMM '17, pages 404–417, Los Angeles, CA, USA, August 2017.
[77] Zilong Wang, Xinchen Wan, Luyang Li, Yijun Sun, Peng Xie, Xin Wei, Qingsong Ning, Junxue Zhang, and Kai Chen. Tassel: Fast, scalable, and accurate rate limiter for RDMA nics. In *Proceedings of the ACM SIGCOMM 2024 Conference*, SIGCOMM '24, pages 1–13, Sydney, NSW, Australia, August 2024.
[78] Danfeng Shan, Wanchun Jiang, and Fengyuan Ren. EDT: Absorbing micro-burst traffic by enhancing dynamic threshold policy of data center switches. In *2015 IEEE Conference on Computer Communications*, INFOCOM '15, pages 118–126, Hong Kong, China, April 2015.
[79] Vamsi Addanki, Maria Apostolaki, Manya Ghobadi, Stefan Schmid, and Laurent Vanbever. ABM: active buffer management in datacenters. In *Proceedings of the ACM SIGCOMM 2022 Conference*, SIGCOMM '22, pages 36–52, Amsterdam, Netherlands, August 2022.
[80] Vamsi Addanki, Wei Bai, Stefan Schmid, and Maria Apostolaki. Reverie: Low pass Filter-Based switch buffer sharing for datacenters with RDMA and TCP traffic. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*, NSDI '24, pages 763–778, Santa Clara, CA, USA, April 2024.
[81] Jiyang Gao, Zhenheng Yang, and Ram Nevatia. RED: Reinforced Encoder-Decoder Networks for Action Anticipation. https://arxiv.org/abs/1707.04818, July 2017. Accessed: 2026-07-03.
[82] Kathleen Nichols and Van Jacobson. Controlling Queue Delay. *Communications of the ACM*, 55(7):42–50, July 2012.
[83] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. MegaScale: Scaling large language model training to more than 10,000 gpus. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*, NSDI '24, pages 1021–1038, Santa Clara, CA, USA, April 2024.
[84] NVIDIA Corporation. Doubling all2all performance with NVIDIA collective communication library 2.12. https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/, 2022. Accessed: 2026-07-03.
[85] NVIDIA Corporation. NVIDIA Spectrum SN5000 Series Switches introduction. https://networking-docs.nvidia.com/sn5000hw/introduction, 2025. Accessed: 2026-07-03.
[86] NVIDIA Corporation. NCCL Environment Variables. https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html, 2026. Accessed: 2026-06-12.
[87] Danfeng Shan, Yunguang Li, Jinchao Ma, Zhenxing Zhang, Zeyu Liang, Xinyu Wen, Hao Li, Wanchun Jiang, Nan Li, and Fengyuan Ren. Occamy: A preemptive buffer management for on-chip shared-memory switches. In *Proceedings of the 20th European Conference on Computer Systems*, EuroSys '25, pages 414–429, Rotterdam, The Netherlands, March 2025.

A Analysis of Large TBO on Initialization

Appendices are supporting material that has not been peer-reviewed.

The following conclusion on the value of the large TBO after the initialization of A2A traffic is theoretically deduced and confirmed by simulations in this section.

Corollary 1. *The value of large TBO on initialization is positively correlated with the number of ports under A2A traffic.*

During the initialization of A2A traffic, packets injected from multiple ingress ports often arrive at the same egress port, resulting in micro-bursts. The intensity and location of micro-bursts are random and irregular, and we focus on the following idealized scenario where the burst is most intense. Specifically, N nodes transmit A2A traffic through a single switch. Packets from $(N-1)$ ingress ports simultaneously inject to 1 egress port, forming an $(N-1)$-to-1 micro-burst. At a time slice, the egress port receives $(N-1)$ packets and forwards 1 packet, resulting in a buffer occupancy buildup of $(N-2)$ packets. At the next time slice, the $(N-1)$-to-1 micro-burst occurs in another egress port, accumulating $(N-2)$ packets again. Meanwhile, the prior port also forwards 1 packet, leaving its port buffer occupancy as $(N-3)$ packets. After all N posts experience such a round of micro-burst, the port buffer occupancy in each port would be 0, 0, 1, 2, ..., $N-3$, $N-2$ packets, and the accumulated TBO is

$$TBO = \frac{(N-1)\cdot(N-2)}{2}\cdot packets \tag{7}$$

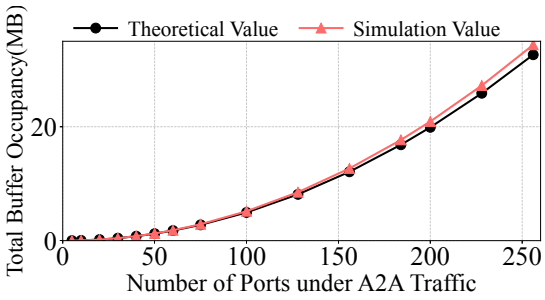


Figure 20: Random collision results in initial large TBO.

A simulation based on NS3 is conducted to verify this theoretical result. We change the number of A2A nodes from 5 to 250, set the packet MTU to 1KB, and initiate A2A QPs in an order that ensures the most intense burst occurs at each port. As shown in Fig. 20, the simulation values closely match the theoretical values calculated by Eq. (7) . This validates that the micro-bursts incurred by A2A traffic indeed generate transient port buffer occupancy buildup, which in turn leads to large total buffer occupancy. Although the TBO is also influenced by the random intensity and location of micro-bursts, these factors do not alter the growth trend of TBO with the number of nodes.

B Proof of Theorem 1

We recall Theorem 1 and provide a proof in this section.

Theorem 1. *Under A2A traffic, due to the Feature 1 of existing CC algorithms, there exists $A(t)$ for which the ingress-egress bandwidth matching cannot be satisfied. This implies that the ingress-egress bandwidth mismatching is inevitable.*

Firstly, under the objective of full egress utilization and the fully-connected characteristic of A2A traffic, the ingress-egress bandwidth matching condition is defined as follows.

Definition 1 (Ingress-Egress Bandwidth Matching). *For any i, j, $QP_{ij}(t) \in S(t)$, there is*

$$\sum_j r_{ij}(t) = \sum_i r_{ij}(t) = b \tag{8}$$

where b is the link bandwidth.

$\sum_i r_{ij}(t) = b$ and $\sum_j r_{ij}(t) = b$ respectively indicate that the bandwidth of the j^{th} egress link and the i^{th} ingress link are full-utilized. Correspondingly, the ingress and egress bandwidth is matched.

Moreover, to facilitate the proof, we assume a single QP exists between an arbitrary ingress port and egress port. Besides, we assume a single-switch topology, so that the ingress port corresponds

to the sender and the egress port corresponds to the receiver. Correspondingly, the sender and receiver are used to denote the ingress and egress ports, respectively. Define the set of all senders as

$$I = \{i \mid i \in [1, N]\},$$

and the set of all receivers as

$$J = \{j \mid j \in [1, N]\}.$$

The set of senders that contain at least one QP in $A(t)$ is defined as

$$I_A = \{i \in I \mid \exists j \in J,\ QP_{ij} \in A(t)\}.$$

Similarly, the set of receivers to which at least one QP in $A(t)$ connects is defined as

$$J_A = \{j \in J \mid \exists i \in I,\ QP_{ij} \in A(t)\}.$$

Moreover, for any subset of senders $X \subseteq I$ and receivers $Y \subseteq J$, we define their neighbor sets according to the QP connection relationship in $A(t)$ as follows. The neighbor set of X, denoted by $NS_A(X)$, is the set of receivers connected by at least one QP from any sender in X, i.e.,

$$NS_A(X) = \{j \in J \mid \exists i \in X,\ QP_{ij} \in A(t)\}.$$

Similarly, the neighbor set of Y, denoted by $NS_A(Y)$, is the set of senders that contain at least one QP towards any receiver in Y, i.e.,

$$NS_A(Y) = \{i \in I \mid \exists j \in Y,\ QP_{ij} \in A(t)\}.$$

Let $I'_A \subseteq I_A$ and $J'_A \subseteq J_A$ denote arbitrary subsets of I_A and J_A, respectively. Their neighbor sets can thus be written as

$$NS_A(I'_A) = \{j \in J \mid \exists i \in I'_A,\ QP_{ij} \in A(t)\},$$
$$NS_A(J'_A) = \{i \in I \mid \exists j \in J'_A,\ QP_{ij} \in A(t)\}.$$

According to Hall's theorem [35], if there exists $I'_A \subseteq I_A$ that satisfy the size of subset I'_A is greater than the size of $NS_A(I'_A)$, i.e., $|I'_A| > |NS_A(I'_A)|$, then the sum of the ingress bandwidth corresponding to senders in I'_A will be greater than the sum of the egress bandwidth corresponding to receivers in $NS_A(I'_A)$, and the ingress bandwidth cannot be fully utilized. Conversely, if there exists $J'_A \subseteq J_A$ that satisfy $|J'_A| > |NS_A(J'_A)|$, then the sum of the egress bandwidth corresponding to receivers in J'_A will be greater than the sum of the ingress bandwidth corresponding to senders in $NS_A(J'_A)$, and the egress bandwidth cannot be fully utilized. In sum, no matter which one of the above conditions occurs, the ingress-egress bandwidth matching cannot be satisfied, leading to prolonged completion time of A2A traffic. Mathematically, Hall's theorem gives a formal expression as follows.

$$\begin{aligned} &((\exists I'_A \subseteq I_A \Rightarrow |I'_A| > |NS_A(I'_A)|) \\ &\Rightarrow (\exists i \in I_A \Rightarrow \textstyle\sum_j r_{ij}(t) \neq b)) \\ \vee\ &((\exists J'_A \subseteq J_A \Rightarrow |J'_A| > |NS_A(J'_A)|) \\ &\Rightarrow (\exists j \in J_A \Rightarrow \textstyle\sum_i r_{ij}(t) \neq b)) \end{aligned} \tag{9}$$

Hall's theorem gives the situation where A2A traffic leads to ingress-egress bandwidth mismatching, and Theorem 1 is proved.

In fact, as long as one of the conditions in Eq. (9) is satisfied, which means either the ingress or egress bandwidth is thoroughly wasted, no CC can achieve ingress-egress bandwidth matching under A2A traffic. However, because of the Feature 1 of existing CC algorithms explained in §3.3, there exist more common conditions that existing CC algorithms cannot achieve ingress-egress bandwidth matching under A2A traffic, although Hall's theorem is not satisfied. As a preliminary, we define the set of all finished QPs as the complement of $A(t)$, denoted by $\widetilde{A}(t)$:

$$\widetilde{A}(t) \subseteq S(t),\ \widetilde{A}(t) \cup A(t) = S(t),\ \widetilde{A}(t) \cap A(t) = \emptyset. \tag{10}$$

Similar to the definition of I_A and J_A, we define

$$I_{\widetilde{A}} = \{i \in I | \exists j \in J, QP_{ij} \in \widetilde{A}(t)\},$$
$$J_{\widetilde{A}} = \{j \in J | \exists i \in I, QP_{ij} \in \widetilde{A}(t)\}$$

More common conditions where existing CC algorithms cannot achieve ingress-egress bandwidth matching under A2A traffic are given below.

(1) $\exists\ \widetilde{A}(t), |I_{\widetilde{A}}| = 1 \wedge |J_{\widetilde{A}}| = 1$: Only one QP finishes.
(2) $\exists\ \widetilde{A}(t), |I_{\widetilde{A}}| = 1 \vee |J_{\widetilde{A}}| = 1$: Some QPs at the same sender or towards the same receiver finish.
(3) $\exists\ \widetilde{A}(t), |I_{\widetilde{A}}| \cdot |J_{\widetilde{A}}| = |\widetilde{A}(t)|$: All QPs connecting the senders in $I_{\widetilde{A}}$ with the receivers in $J_{\widetilde{A}}$ finish.

Condition 2 has been discussed in §3.3. In fact, both conditions 1 and 2 are special cases of condition 3. Here, we prove condition 3 as follows.

Proof. On the one hand, when $\widetilde{A}(t)$ satisfies $|I_{\widetilde{A}}| + |J_{\widetilde{A}}| > N$, it can be proved that this case satisfies Hall's theorem. Specifically, when $|I_{\widetilde{A}}| \cdot |J_{\widetilde{A}}| = |\widetilde{A}(t)|$, $\widetilde{A}(t)$ can be described as

$$\widetilde{A}(t) = \{QP_{ij}(t) \mid \forall i \in I_{\widetilde{A}},\ \forall j \in J_{\widetilde{A}}\}$$

According to Eq. (10), we have

$$\forall i \in I_{\widetilde{A}},\ \nexists j \in J_{\widetilde{A}} \text{ s.t. } QP_{ij}(t) \in A(t) \tag{11}$$

In other words, for all $i \in I_{\widetilde{A}}$ and $QP_{ij}(t) \in A(t)$, we have $\forall j \notin J_{\widetilde{A}}$. Subsequently, the neighbor set of $I_{\widetilde{A}}$ can be expressed as

$$\begin{aligned} NS_A(I_{\widetilde{A}}) &= \{j \in J \mid \exists i \in I_{\widetilde{A}},\ QP_{ij} \in A(t)\} \\ &= \{j \in J \mid \forall j \notin J_{\widetilde{A}}\} = J - J_{\widetilde{A}} \end{aligned} \tag{12}$$

Therefore, we have $|NS_A(I_{\widetilde{A}})| = |J| - |J_{\widetilde{A}}| = N - |J_{\widetilde{A}}|$. Because of $|I_{\widetilde{A}}| > N - |J_{\widetilde{A}}|$, we can get $|I_{\widetilde{A}}| > |NS_A(I_{\widetilde{A}})|$. This satisfies the first condition in Eq. (9), and thus, the ingress-egress bandwidth matching cannot be satisfied.

On the other hand, when $\widetilde{A}(t)$ satisfies $|I_{\widetilde{A}}| + |J_{\widetilde{A}}| <= N$, we prove condition 3 by contradiction. We assume that CC algorithms can adjust the average calculated rate of each QP to make $A(t + \delta t)$ satisfy ingress-egress bandwidth matching under Feature 1. According to Definition 1, we have the following equations.

$$\forall i \in I_{\widetilde{A}}, \sum_{j \notin J_{\widetilde{A}}} r_{ij}(t+\delta t) = b \tag{13}$$

$$\forall j \in J_{\widetilde{A}}, \sum_{i \notin I_{\widetilde{A}}} r_{ij}(t+\delta t) = b \tag{14}$$

Moreover, because all QPs in $\widetilde{A}(t)$ have finished, we have

$$\forall i \in I_{\widetilde{A}}\, j \in J_{\widetilde{A}}, \sum r_{ij}(t) = \sum r_{ij}(t+\delta t) = 0 \tag{15}$$

By combining Eq. (13), Eq. (14), and Eq. (15), we can get

$$\forall i \in I_{\widetilde{A}} \sum_{j \in J} r_{ij}(t+\delta t) = b \tag{16}$$

$$\forall j \in J_{\widetilde{A}} \sum_{i \in I} r_{ij}(t+\delta t) = b \tag{17}$$

Under the assumption that $A(t+\delta t)$ satisfies ingress-egress bandwidth matching, we have

$$\begin{aligned}\sum_{i \notin I_{\widetilde{A}}, j \notin J_{\widetilde{A}}} r_{ij}(t+\delta t) &= \sum_{i \in I, j \in J} r_{ij}(t+\delta t) - \sum_{i \in I_{\widetilde{A}}, j \in J} r_{ij}(t+\delta t) \\ &\quad - \sum_{i \in I, j \in J_{\widetilde{A}}} r_{ij}(t+\delta t) + \sum_{i \in I_{\widetilde{A}}, j \in J_{\widetilde{A}}} r_{ij}(t+\delta t) \\ &= (N - |I_{\widetilde{A}}| - |J_{\widetilde{A}}|) \cdot b\end{aligned} \tag{18}$$

Knowing that $S(t)$ satisfies ingress-egress bandwidth matching in a stable state before any QP finishes, we can get

$$\begin{aligned}\sum_{i \notin I_{\widetilde{A}}, j \notin J_{\widetilde{A}}} r_{ij}(t) &= \sum_{i \in I, j \in J} r_{ij}(t) - \sum_{i \in I_{\widetilde{A}}, j \in J} r_{ij}(t) \\ &\quad - \sum_{i \in I, j \in J_{\widetilde{A}}} r_{ij}(t) + \sum_{i \in I_{\widetilde{A}}, j \in J_{\widetilde{A}}} r_{ij}(t) \\ &= (N - |I_{\widetilde{A}}| - |J_{\widetilde{A}}|) \cdot b + \sum_{i \in I_{\widetilde{A}}, j \in J_{\widetilde{A}}} r_{ij}(t) \\ &> \sum_{i \notin I_{\widetilde{A}}, j \notin J_{\widetilde{A}}} r_{ij}(t+\delta t)\end{aligned} \tag{19}$$

As shown in Equation (19), in order to achieve ingress-egress bandwidth matching, CC must proactively reduce the rate of QPs in region ($i \notin I_{\widetilde{A}}, j \notin J_{\widetilde{A}}$), which violates the Feature 1. Therefore, when $\widetilde{A}(t)$ satisfies $|I_{\widetilde{A}}| + |J_{\widetilde{A}}| <= N$, the ingress-egress bandwidth matching is not satisfied.

In summary, when $A(t)$ satisfies condition 3, ingress-egress bandwidth matching cannot be satisfied under A2A traffic. □

C Configurations

We summarize the key experimental parameters used in our evaluation and explain why they are representative of modern AI cluster deployments. Notably, the large TBO problem is observed in our real-world production networks and has also been reported in Meta [10]. The three observed counterintuitive phenomena help to uncover the underlying reasons, but they do not necessarily occur under every network configuration.

Network Topology. We use a 3-layer fat-tree topology, with 128 nodes/128-GPUs in total and 4 nodes/GPUs per rack/ToR, as shown in Fig. 1(a). This setting is chosen to model one rail of the multi-rail network [83]. In the multi-rail fabric, inter-rail traffic forwarding relies on NCCL's PXN mechanism [84], which routes data via NVSwitch onto the GPU of the designated rail, while intra-rail traffic forwarding still relies on the traditional Clos fabric. Accordingly, it is structurally sufficient to evaluate Odin within a single rail, where each host just uses one NIC and one GPU. Moreover, 128 nodes are large enough to expose the A2A-induced large TBO problem, while production AI clusters would amplify the problem. In addition, our testbed employs 8 nodes connected by one switch, corresponding to a local rack/ToR-domain portion of one rail.

Switch Configuration. We configure each switch with 128 400Gbps ports, following NVIDIA Spectrum SN5600 [85], which provides 128 400Gbps ports and 128MB buffer. We allocate 82MB as the shared buffer and reserve the remaining buffer for PFC headroom. PFC is enabled, and the PFC threshold is dynamic according to [32]. In addition, although we take the dynamic buffer allocation as an example, the A2A-induced large TBO problem would also occur under static-buffer configuration, since static allocation cannot eliminate the simultaneous accumulation of many small queues, nor does it address NIC-side rate inflation or ingress-egress bandwidth mismatch. However, the PFC-before-ECN phenomenon would disappear because the PFC threshold is not directly reduced by total buffer occupancy.

QP Scale. We set 10 QPs or 5 QPs per source-destination pair. This setting follows practical GPU-communication tuning. For instance, NCCL exposes the *NCCL_IB_QPS_PER_CONNECTION* field, which controls the number of QPs per source-destination connection [86]. NCCL states that multiple QPs can be useful on multi-level fabrics to improve routing entropy, with a valid range of $(1, 128)$. Meta applies 4 QPs for All-to-All communication in its Ranking models and uses 16 QPs for LLM workloads [10]. Therefore, 10 QPs or 5 QPs is a moderate setting.

All-to-All traffic pattern. We evaluate Odin under realistic MoE-based A2A traffic, NCCL-based asynchronous A2A traffic, and constructed synchronous A2A traffic. Specifically, the MoE-based A2A traffic is generated using SimAI based on the *A100_Mixtral_MoE* model [46], where A2A communication appears as cyclic bursts. The asynchronous A2A traffic follows NCCL [47], where QPs are initiated in batches of up to 8 QPs with random initiation intervals of up to $50\mu s$. Moreover, the synchronous A2A traffic is constructed to present the A2A-induced issues and facilitate understanding of the behavior of different CC algorithms. The short flow is sometimes introduced to allow us to more clearly identify the underlying reasons for the A2A-induced large TBO problem.

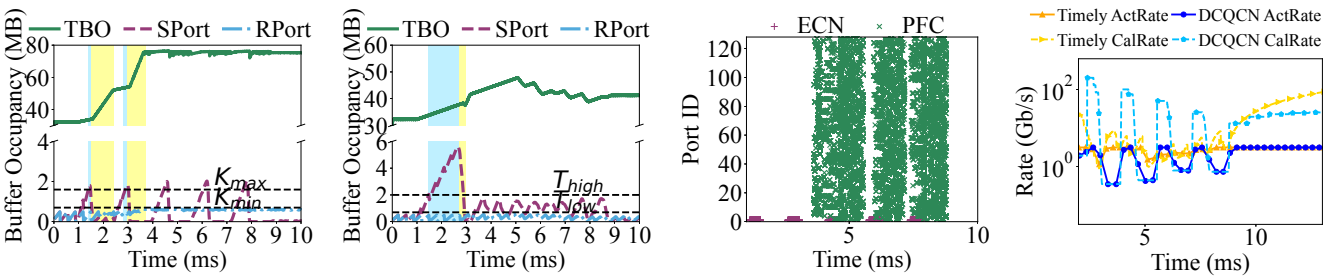
D Additional Experiments

D.1 Single QP per Source-Destination Pair Scenario

The experiments in Fig. 2 are repeated with a single QP per source-destination pair. As shown in Fig. 21, similar counterintuitive phenomena are also observed. For Phenomenon 1, PFC still triggers before ECN on almost all switch ports in Fig. 21(c). For Phenomenon 2, as shown in the blue regions of Fig. 21(a) and Fig. 21(b), the phenomenon is alleviated for DCQCN because reducing the number of QPs significantly mitigates rate inflation and sparse rate adjustment. However, Timely still takes several milliseconds to decrease the sending rate for buffer occupancy elimination, since it decreases rates conservatively under mild congestion [15]. For Phenomenon 3, as shown in the yellow regions of Fig. 21(a) and Fig. 21(b), the buildup of TBO is still accelerated when DCQCN takes effect, while the phenomenon is alleviated for Timely since its rate inflation issue is less severe under the single-QP setting, as shown in Fig. 21(d). Overall, although some phenomena are alleviated under the conservative single-QP setting, the underlying issues still hold under A2A traffic.

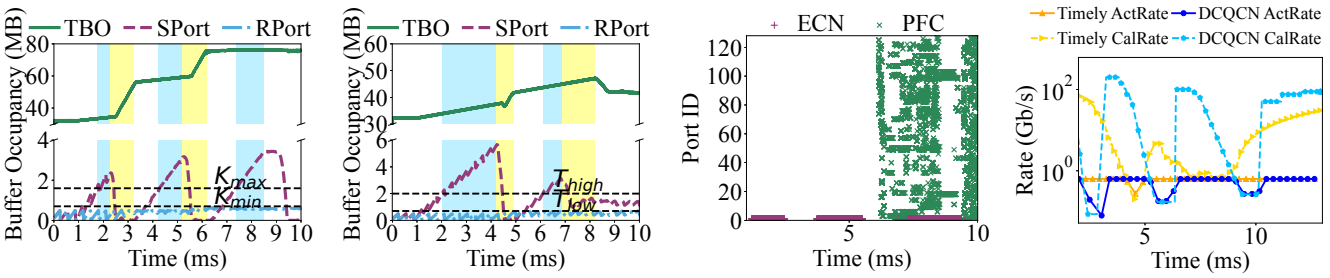
D.2 Asynchronous Start Scenario

The experiment in Fig. 2 is repeated by making the start time of each flow a uniform random value in the range $(0, 1)\mu s$. As shown in Fig. 22, the three counterintuitive phenomena observed in §2.2.1



(a) Buffer occupancy of DCQCN (b) Buffer occupancy of Timely (c) PFC and ECN of DCQCN (d) Rate Adjustment

Figure 21: Counterintuitive Phenomena under A2A Traffic, under 1 QP per source-destination pair setting.



(a) Buffer occupancy of DCQCN (b) Buffer occupancy of Timely (c) PFC and ECN of DCQCN (d) Rate Adjustment

Figure 22: Counterintuitive Phenomena under A2A Traffic, under asynchronous start setting.

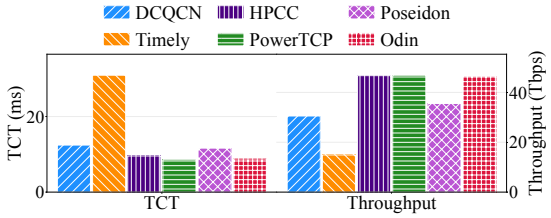


Figure 23: Pure All-Reduce Results.

still persist under this asynchronous start setting. Together with the asynchronous results in Table 2, where flows' start times are in the range of $(0, 50)\mu s$, this result confirms that the motivating phenomena are not artifacts of synchronized flow starts.

D.3 Pure All-Reduce Results

We evaluate Odin and other comparisons under pure all-reduce traffic. The all-reduce traffic is constructed according to [87], where flows are generated based on the double binary tree algorithm, and all flows have an identical size. We report the TCT of All-Reduce traffic and the switch throughput. As shown in Fig. 23, Odin achieves similar performance to existing INT-based CC algorithms, and performs better than DCQCN and Timely.